

School of Computer Science Engineering and Information Systems
VIT, Vellore

DATABASE SYSTEMS

PROJECT REPORT

"Vehicle Rental Management System"

A Relational Database with Triggers, Transactions and a Web Application

Submitted by
Sarvesh R
24BYB0081

Academic Year: 2026–2027

Full code: GitHub Link

CERTIFICATE

This is to certify that the project report entitled "Vehicle Rental Management System" is a bona fide record of the work carried out by Sarvesh R 24BYB0081 in partial fulfilment of the requirements of the course Database Systems at the School of Computer Science Engineering and Information Systems, VIT, Vellore, during the academic year 2026–2027.

Faculty Guide: Dr. THANGA MARIAPPAN L

Date: 02/11/2026

DECLARATION

I/We hereby declare that this project report is the result of my/our own work, carried out under the guidance of the faculty named in the certificate. It has not been submitted to any other course or institution. Where ideas, text, figures or code from other sources have been used, they have been acknowledged in the References. All data used in the project is fictitious and contains no real personal or financial information.

Signature:



Name: Sarvesh R

Date: 02/11/2026

ACKNOWLEDGEMENT

I/We express sincere thanks to Dr. THANGA MARIAPPAN L, the School of Computer Science Engineering and Information Systems and VIT, Vellore for the guidance, laboratory facilities and encouragement that made this project possible. I/We also thank family and friends for their support, and the developers of the open-source tools used: Oracle AI Database 26ai Free, Python 3.12, Flask, python-oracledb, Jinja2, HTML5 and CSS3.

ABSTRACT

Vehicle rental enterprises handle high-value physical assets and continuous operational transactions: tracking fleet availability across branches, validating customer driving credentials, scheduling reservations, preventing double-booking collisions, enforcing maintenance quarantines, and reconciling multi-mode payments. In this domain, business logic discrepancies directly induce contractual breaches and financial loss. This project presents an enterprise-grade Vehicle Rental Management System (VRMS) built to demonstrate the core principles of relational database management natively within Oracle AI Database 26ai Free.

The relational schema comprises eight normalized tables (branch, category, customer, vehicle, booking, payment, maintenance, and bk_log) governed by primary key, foreign key, NOT NULL, UNIQUE, CHECK, and DEFAULT constraints. Controlled denormalization is applied deliberately for booking totals, which are autonomously calculated from category daily rates via a BEFORE INSERT trigger (trg_bk_bi) that also prohibits booking vehicles under active maintenance. An AFTER UPDATE trigger (trg_bk_au) synchronizes vehicle availability states (AVAILABLE and RENTED) and records an immutable audit history in bk_log. Stored functions (f_bal, f_avail) and stored procedures (p_book, p_start, p_return, p_pay, p_dues) encapsulate all transactional workflows directly in PL/SQL. An explicit cursor with %ROWTYPE generates comprehensive outstanding dues reports. The system provides sixteen verified SQL queries (covering multi-table joins, aggregations, HAVING filters, subqueries, set operations, and views). A Python Flask web application connects via the thin oracledb driver to provide dedicated Staff and Manager portals with zero client-side ORM. Twenty-eight comprehensive test cases verify full functional correctness and constraint enforcement.

Keywords: *Database Management System, ER Model, Oracle 26ai, SQL, Normalization, PL/SQL, Transactions, Triggers, Vehicle Rental, Python Flask*

TABLE OF CONTENTS

Chapter / Section	Page
Certificate	2
Declaration	2
Acknowledgement	3
Abstract	3
List of Figures	4
List of Tables	4
List of Abbreviations	6
Chapter 1 – Introduction	7
Chapter 2 – Requirements Analysis	9
Chapter 3 – DBMS Concepts and Architecture	11
Chapter 4 – Conceptual Database Design	12
Chapter 5 – Relational Database Design	14
Chapter 6 – Functional Dependencies and Normalization	17
Chapter 7 – SQL Implementation	20
Chapter 8 – Advanced Database Operations	28
Chapter 9 – Query Processing and Optimization	33
Chapter 10 – Database Security, Transactions and Recovery	34
Chapter 11 – Application Development and Connectivity	36
Chapter 12 – Testing and Results	38
Chapter 13 – Conclusion and Future Enhancements	41
References	42
Appendix A – SQL Scripts	43
Appendix B – Sample Data and Screenshots	46
Appendix C – Sample Test Case Table	48

LIST OF FIGURES

Figure Caption	Page
Figure 2.1: Use-case diagram of the Vehicle Rental Management System	10
Figure 4.1: Entity-Relationship diagram (diamonds = relationships; UK = unique key)	13
Figure 5.1: Relational schema diagram with primary and foreign key links	15
Figure 11.1: Three-tier architecture of the Vehicle Rental Management System	36
Figure 11.2: Web application: role-based authentication portal	37

Figure 11.3: Staff portal: operations dashboard and live fleet metrics 37
Figure 11.4: Staff portal: new reservation booking with trigger validation 38
Figure 11.5: Staff portal: active rental tracking, trip dispatch, and return 38
Figure 11.6: Manager portal: executive KPI dashboard and fleet revenue 38
Figure 11.7: Manager portal: outstanding receivables audit report via cursor 38
Figure B.1: Manager portal: interactive database tables and data dictionary inspector 48

LIST OF TABLES

Table Title	Page
Table 2.1: User roles and responsibilities	9
Table 2.2: Functional requirements and implementing component	9
Table 2.3: Hardware and software requirements	10
Table 2.4: Key use-case descriptions	10
Table 3.1: File system versus DBMS in the vehicle rental context	11
Table 3.2: SQL sub-languages and their use in the project	12
Table 4.1: Entities and their roles	12
Table 4.2: Relationships and cardinality	12
Table 5.1: Keys of each relation	14
Table 5.2: Integrity constraints used	15
Table 5.3: Data dictionary: BRANCH	16
Table 5.4: Data dictionary: CATEGORY	16
Table 5.5: Data dictionary: CUSTOMER	16
Table 5.6: Data dictionary: VEHICLE	16
Table 5.7: Data dictionary: BOOKING	16
Table 5.8: Data dictionary: PAYMENT	17
Table 5.9: Data dictionary: MAINTENANCE	17
Table 5.10: Data dictionary: BK_LOG	17
Table 6.1: BCNF check: every non-trivial determinant must be a superkey	19
Table 6.2: Before: flat relation (selected columns)	20
Table 6.3: After: normalized BOOKING and VEHICLE rows	20
Table 7.1: Functions used in the project SQL	21
Table 7.2: Join types demonstrated	21
Table 7.3: Output of the view query (v_open_bookings)	22
Table 7.4: Constraint demonstration (actual Oracle error responses)	22
Table 7.5 to 7.20: Outputs of Queries 1 to 16	23-28
Table 8.1: Procedures in the running implementation	28
Table 8.2: Seeded booking totals versus category tariffs	29
Table 8.4: ACID properties and evidence	31
Table 8.5: Savepoint demonstration values	32
Table 8.6: Concurrency and row-level locking experiment	32
Table 9.1: Automatic and explicit indexes in the database	33
Table 9.2: Execution plans (EXPLAIN PLAN actual output)	33
Table 9.3: Query 4 plan with foreign key index	34
Table 9.4: Effect of the composite index on query latency	34

Table 10.1: Permitted and denied operations (security test results)	36
Table 11.1: Technology stack	36
Table 11.2: CRUD workflows and application routes	37
Table 12.1: Test cases and actual results (28 tests)	38
Table 12.2: Manual verification of query results	40
Table B.1 to B.8: Sample data for all 8 tables	46-47
Table C.1: Sample test cases summary	48

LIST OF ABBREVIATIONS

- **DBMS –** Database Management System
- **RDBMS –** Relational Database Management System
- **SQL –** Structured Query Language
- **DDL –** Data Definition Language
- **DML –** Data Manipulation Language
- **DQL –** Data Query Language
- **DCL –** Data Control Language
- **TCL –** Transaction Control Language
- **PL/SQL –** Procedural Language extension to SQL (Oracle)
- **ER –** Entity–Relationship
- **ACID –** Atomicity, Consistency, Isolation, Durability
- **CRUD –** Create, Read, Update, Delete
- **PK / FK / UQ –** Primary Key / Foreign Key / Unique constraint
- **BCNF –** Boyce–Codd Normal Form
- **3NF –** Third Normal Form
- **FD –** Functional Dependency
- **DL –** Driving License Number
- **API –** Application Programming Interface
- **REST –** Representational State Transfer
- **JSON –** JavaScript Object Notation
- **PDB –** Pluggable Database (Oracle Multitenant Architecture)
- **VRMS –** Vehicle Rental Management System

CHAPTER 1: INTRODUCTION

1.1 Background

Vehicle rental enterprises handle a continuous stream of critical operational and financial transactions: onboarding customers with verified identity credentials, allocating physical vehicles across distributed branch locations, managing dynamic daily rental tariffs, executing reservation bookings across future date intervals, dispatching vehicle handovers, inspecting returns, tracking scheduled fleet servicing, and recording multi-channel payments. In this environment, an operational mistake is not merely a software defect; it represents asset damage, legal liability, or direct financial loss. A vehicle double-booked for overlapping dates, an unrecorded handover of a luxury car, a vehicle dispatched while quarantined in service, or an untracked payment variance directly impairs business viability.

A Relational Database Management System (RDBMS) provides the natural foundation for this mission-critical domain. It guarantees data consistency by storing every operational fact exactly once, enforces domain and business integrity declaratively through constraints, guarantees that multi-step operations execute under ACID transactional semantics, and provides multi-version concurrency control (MVCC). This project, the Vehicle Rental Management System (VRMS), implements those core database principles in a realistic fleet rental enterprise using Oracle AI Database 26ai Free (SQL and PL/SQL) combined with a Python Flask web frontend.

1.2 Problem Statement

Spreadsheet-based, flat-file, or uncoordinated paper-based systems commonly used in regional rental agencies suffer from the following severe operational vulnerabilities:

- **Redundancy and Update Anomalies:** Customer, category, and branch attributes repeated across each booking record lead to inconsistent data when customer phone numbers or branch managers change.
- **Double-Booking and Temporal Collisions:** Concurrent reservations for the same vehicle across overlapping dates occur when availability checks are handled manually or outside atomic database locks.
- **Unenforced Safety Rules:** Vehicles flagged for critical mechanical maintenance or scheduled overhaul can be mistakenly reserved or dispatched, risking customer safety.
- **Manual Tariff and Payment Errors:** Manual calculation of multi-day rental tariffs and payment reconciliation results in arithmetic errors and revenue leakage.
- **Absence of Traceability and Audit Trail:** Without autonomous database triggers logging state transitions (BOOKED to ACTIVE to RETURNED), it is impossible to determine who authorized an operation or when a vehicle changed custody.

The primary stakeholders affected by these issues are retail customers (who face reservation cancellations or unsafe vehicles), branch staff (who manage vehicle dispatches and payment collection), and fleet managers/auditors (who require verifiable revenue audits and maintenance tracking).

1.3 Objectives

1. Design and normalize an 8-table relational schema with appropriate Primary Keys, Foreign Keys, NOT NULL, UNIQUE, CHECK, and DEFAULT constraints, populated with at least 10 meaningful records per table.

2. Enforce 100% of the operational business rules directly inside Oracle Database using PL/SQL stored procedures (p_book, p_start, p_return, p_pay, p_dues) and deterministic stored functions (f_bal, f_avail).

3. Implement row-level database triggers (trg_bk_bi, trg_bk_au) to autonomously compute rental tariffs, block bookings for vehicles under service, synchronize vehicle availability states, and populate an immutable audit ledger (bk_log).

4. Demonstrate explicit multi-record cursor processing with %ROWTYPE (p_dues) to generate automated receivables recovery reports.

5. Implement and verify 16 comprehensive SQL queries covering multi-table joins, aggregations, HAVING clauses, correlated/uncorrelated subqueries, set operations, single-row string functions, date functions, and views.

6. Develop an interactive, 100% offline Python Flask web application connecting via python-oracledb Thin mode, featuring role-based Staff and Manager portals and an interactive database tables browser.

1.4 Scope

Included: Ten branch locations across Tamil Nadu and major technology hubs, ten distinct vehicle categories with daily tariffs, ten registered customers with driving credentials, ten fleet vehicles with operational status tracking (AVAILABLE, RENTED, SERVICE), twelve multi-state bookings (BOOKED, ACTIVE, RETURNED, CANCELLED), multi-mode payments (CASH, UPI, CARD), fleet maintenance logs, an autonomous trigger-populated audit trail, role-based web portals (Staff and Manager), and whitelisted data dictionary inspector. Not included: External credit-card payment gateway gateways, automated SMS dispatchers, and production GPS telematics integration.

1.5 Existing System

The reference 'existing system' is a spreadsheet or uncoordinated departmental ledger where each reservation row repeats customer names, contact numbers, branch addresses, vehicle models, and daily rates. Data updates require edits across hundreds of redundant cells, vehicle maintenance schedules are decoupled from rental desk operations, and historical balance revisions are untraceable. Chapter 6 illustrates this as a concrete universal relation and quantifies the anomalies it produces.

1.6 Proposed System

The proposed system establishes a strict, database-centric three-tier architecture. Each operational entity is stored once in a normalized relational schema (BCNF / 3NF). Business logic is pushed entirely into the Oracle Database tier via PL/SQL routines, ensuring that business rules remain inviolable regardless of whether commands originate from web forms, SQL Developer, or batch scripts. The front end consists of a responsive Python Flask application providing intuitive, role-separated portals for Branch Staff and Fleet Managers.

1.7 Organization of the Report

Chapter 2 presents the requirements analysis and stakeholder models. Chapter 3 examines foundational DBMS concepts and architecture. Chapters 4 and 5 detail the conceptual ER model and relational schema. Chapter 6 analyzes functional dependencies and BCNF normalization. Chapters 7 and 8 document the SQL query suite, PL/SQL stored routines, triggers, cursors, and ACID transactions. Chapter 9 covers query optimization and indexing. Chapter 10 addresses security, backup, and recovery. Chapter 11 describes the web application implementation. Chapter 12 details empirical testing and 28 test cases. Chapter 13 concludes with learning outcomes and future enhancements. Appendices provide complete SQL scripts, sample datasets, and test summaries.

CHAPTER 2: REQUIREMENTS ANALYSIS

2.1 Stakeholders and User Roles

Table 2.1: User roles and responsibilities

Role	Responsibilities in the System
Retail Customer	Browses vehicle availability across branches, places reservations, takes vehicle delivery, makes payments, and receives trip receipts.
Branch Staff / Desk Agent	Validates customer driving credentials, executes bookings (p_book), dispatches vehicle pickup (p_start), processes returns (p_return), and records payments (p_pay).
Fleet Manager / Auditor	Monitors branch inventory and vehicle utilization, reviews maintenance expenses, tracks outstanding receivables via cursor audit (p_dues), and reviews immutable audit logs (bk_log).
Database Administrator (DBA)	Manages schema DDL, tunes indexes, configures tablespaces, monitors lock contention, and manages physical/logical backups via RMAN and Data Pump.

2.2 Functional Requirements

Table 2.2: Functional requirements and implementing component

ID	Requirement	Operational Description	Implementing Component
FR1	Customer Onboarding	Register customer with unique phone, email, and verified driving license	customer table, UK constraints
FR2	Vehicle Cataloging	Catalog vehicles categorized with daily rates and branch assignments	category, branch, vehicle tables
FR3	Temporal Availability Check	Verify vehicle is not in SERVICE and has no overlapping reservations	f_avail(p_v, p_s, p_e)
FR4	Automated Booking Creation	Reserve vehicle using sequence seq_bk; compute total automatically	p_book, seq_bk, trg_bk_bi
FR5	Service Quarantine Protection	Prohibit reservation or dispatch of vehicles undergoing maintenance	trg_bk_bi (ORA-20005)
FR6	Trip Dispatch & State Sync	Transition BOOKED to ACTIVE; lock vehicle status to RENTED automatically	p_start, trg_bk_au
FR7	Vehicle Return Processing	Transition ACTIVE to RETURNED; restore vehicle to AVAILABLE status	p_return, trg_bk_au
FR8	Audit Trail Logging	Record immutable historical log of every booking status transition	trg_bk_au -> bk_log (seq_log)
FR9	Payment Collection & Validation	Record payments; reject amounts exceeding outstanding balance	p_pay, seq_pay, f_bal
FR10	Outstanding Receivables Audit	Iterate over open bookings to report customers with pending balances	p_dues (explicit cursor)
FR11	Analytical SQL Reporting	Execute 16 verified SQL queries for management insights	Queries 1 to 16
FR12	Whitelisted Table Inspection	Direct interactive inspection of live database tables and constraints	Manager portal /tables route

2.3 Non-Functional Requirements

- **Data Integrity & Consistency:** Enforced via declarative database constraints (PK, FK, UK, CHECK) and stored procedures with atomic commit and rollback.
- **Auditability & Non-Repudiation:** Every status mutation is autonomously intercepted by trg_bk_au and recorded in bk_log with old status, new status, timestamp, and sequence ID.
- **Robust Error Handling:** Meaningful application error numbers (ORA-20001 to ORA-20005) are raised in PL/SQL and translated into user-friendly alerts in the UI.
- **Zero Client-Side ORM:** Strict boundary enforcement where Python issues direct parameterized SQL and executes stored procedures, avoiding synchronization lag.
- **Zero-Dependency Offline Operation:** Frontend styling is 100% self-contained CSS3 without CDN links, ensuring offline air-gapped operation.

2.4 Hardware and Software Requirements

Table 2.3: Hardware and software requirements

Item	Specification / Environment
Operating System	Microsoft Windows 10/11 (64-bit), Linux, or macOS
Database Engine	Oracle AI Database 26ai Free (Port 1521, Service: FREEPDB1)
Application Backend	Python 3.12 (Standard Library: http, json, socket)
Database Driver	python-oracledb 26.0.1 (Thin Mode, no Oracle Instant Client required)
Frontend Web Stack	Flask 3.1.3, Jinja2 Templates, Pure Responsive CSS3
Development Tools	Oracle SQL Developer 26.2.0, Microsoft VS Code, PowerShell 7
Minimum Hardware	4 GB RAM (8 GB recommended for Oracle DB), 15 GB free disk, Dual-Core 2.0 GHz CPU

2.5 Feasibility Study

Technical Feasibility: Oracle AI Database 26ai Free runs natively as a Windows service and supports all required advanced database constructs (triggers, sequences, stored packages, %ROWTYPE cursors). python-oracledb operates seamlessly in Thin mode. Operational Feasibility: The dual-portal interface separates operational workflows for Branch Staff and Fleet Managers, requiring zero database command knowledge for desk users. Economic Feasibility: All software components utilized (Oracle Free, Python, Flask) are open-source or free community releases, incurring zero licensing expenditure.

2.6 Use-Case Diagram and Descriptions

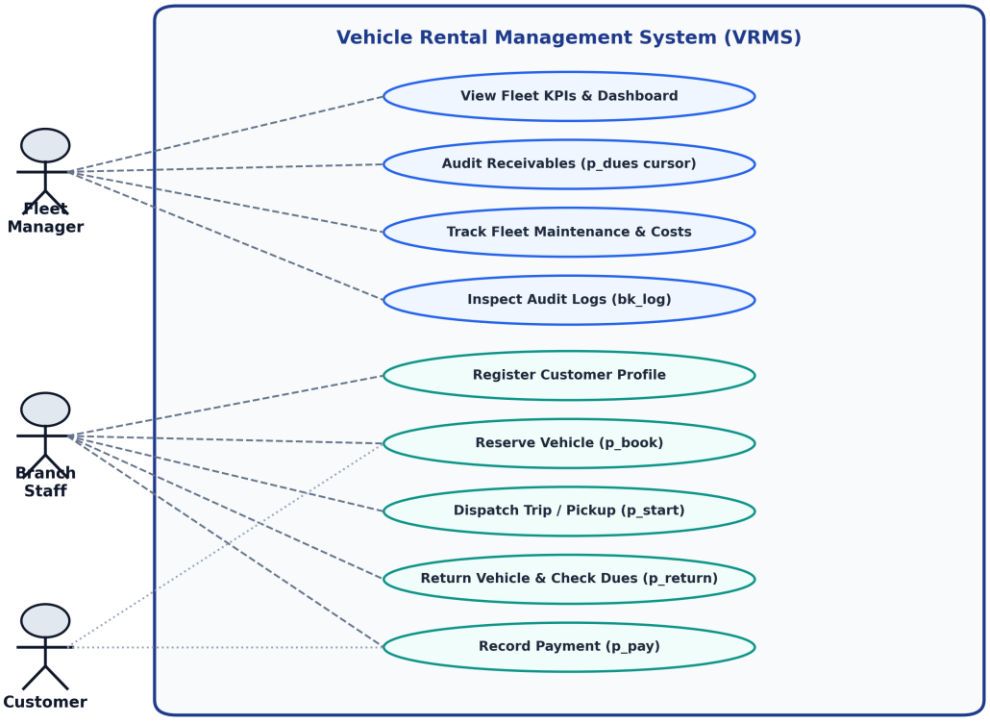


Figure 2.1: Use-case diagram of the Vehicle Rental Management System (VRMS)

Table 2.4: Key use-case descriptions

Use Case	Actors	Main Operational Flow	Alternative / Exception Flow
Reserve Vehicle	Customer / Staff	Select customer, vehicle, dates; verifies f_avail; inserts BOOKED row; trg_bk_bi computes total.	Vehicle under SERVICE or date collision: ORA-20001 raised; transaction rolled back.
Dispatch Trip	Branch Staff	Verifies customer arrival; executes p_start; updates status to ACTIVE; trg_bk_au locks vehicle to RENTED.	Booking not BOOKED: ORA-20002 raised; operation rejected.
Return Vehicle	Branch Staff	Executes p_return; sets status to RETURNED; trg_bk_au unlocks vehicle to AVAILABLE; calls f_bal.	Booking not ACTIVE: ORA-20003 raised; return rejected.
Collect Payment	Staff / Customer	Customer pays via CASH/UPI/CARD; p_pay verifies amount <= balance; inserts row via seq_pay.	Amount <= 0 or exceeds balance: ORA-20004 raised.
Audit Receivables	Fleet Manager	Manager requests dues report; p_dues cursor iterates over open bookings; aggregates total dues.	No open dues: reports 0 overdue balances.

CHAPTER 3: DBMS CONCEPTS AND ARCHITECTURE

3.1 Introduction to DBMS

A Database Management System (DBMS) is specialized system software designed to define, construct, manipulate, and safeguard persistent datasets. In a Relational DBMS (RDBMS), data is conceptually structured into mathematical relations (tables) comprising tuples (rows) and attributes (columns). Relationships between entities are established through key values rather than physical disk pointers. The core benefits include declarative data integrity, reduction of redundancy, concurrent multi-user execution via ACID transactions, standardized query execution (SQL), and enterprise-grade backup/recovery mechanisms.

3.2 File System versus DBMS

Table 3.1: File system versus DBMS in the vehicle rental context

Evaluation Aspect	Traditional File / Spreadsheet System	Oracle RDBMS (This Project)
Data Redundancy	Branch and customer data duplicated across every reservation spreadsheet.	Stored exactly once in normalized tables; referenced via foreign keys.
Data Consistency	Manual cross-sheet synchronization; prone to conflicting values.	Enforced through PK, FK, NOT NULL, UNIQUE, CHECK, and triggers.
Concurrent Sharing	Coarse file locking; simultaneous edits cause data loss or lock conflicts.	Row-level locking (SELECT FOR UPDATE) and multi-version concurrency control.
Data Security	Coarse OS file permissions; no field-level or role-based restrictions.	Oracle database schemas, roles, and granular privilege grants.
Transaction Recovery	Manual file backups; power loss or process crash corrupts data.	Write-ahead logging (Redo logs) and undo tablespaces for atomic rollback.

3.3 Three-Schema Architecture

The ANSI/SPARC Three-Schema Architecture separates database systems into three distinct abstraction tiers:

- **External Schema (View Level):** Customized user interfaces presenting only role-relevant data. The Staff portal displays vehicle availability calendars and customer check-in forms. The Manager portal displays financial aggregate summaries, receivables audits, and maintenance schedules.
- **Conceptual Schema (Logical Level):** The global relational schema defining the 8 normalized tables, data types, primary/foreign key relationships, and enterprise business rules independent of storage mechanics.
- **Internal Schema (Physical Level):** The physical layout of the database within Oracle tablespaces, data files, B*Tree index structures, and row formats stored on disk.

3.4 Data Independence

Logical Data Independence is achieved because the Python Flask web application interacts with the database through SQL views (such as v_open_bookings) and parameterized stored procedures. Adding an operational column or creating audit tables does not break existing application routes. Physical Data

Independence is demonstrated because B*Tree indexes (such as idx_veh_cid_bid) can be added, rebuilt, or relocated without modifying the application SQL queries.

3.5 Data Models

Earlier hierarchical and network data models relied on rigid physical tree pointers and lacked ad-hoc query capabilities. Document and NoSQL databases provide flexible schema nesting but sacrifice multi-table foreign key enforcement and declarative ACID guarantees. The Relational Model was chosen because vehicle rental operations are inherently structured, require strict referential integrity across 8 entities, and demand deterministic financial transactions.

3.6 Database Languages

Table 3.2: SQL sub-languages and their use in the project

Language	Full Form	Purpose	Project Implementation Examples
DDL	Data Definition Language	Defines and alters schema structures	CREATE TABLE, CREATE SEQUENCE, CREATE TRIGGER, DROP TABLE in 01_schema_data.sql
DML	Data Manipulation Language	Inserts, modifies, and deletes data	INSERT sample data, UPDATE status, DELETE customer in 01_schema_data.sql
DQL	Data Query Language	Retrieves data matching criteria	16 comprehensive SQL queries in 03_queries_demo.sql
DCL	Data Control Language	Manages user privileges and roles	GRANT CONNECT, RESOURCE to rental; proposed role-based privilege model
TCL	Transaction Control Language	Controls atomic transaction states	COMMIT, ROLLBACK, SAVEPOINT executed in stored procedures and test cases

3.7 Users and Database Administrator

In this project, user roles reflect operational banking/rental hierarchies: End Users (Customers and Desk Staff) utilize the web interface; Application Programmers develop Python Flask endpoints and Jinja2 views; Database Designers formulate the ER diagram and normalized schema; and the Database Administrator (DBA) establishes the Oracle pluggable database, compiles PL/SQL packages, and oversees data integrity.

CHAPTER 4: CONCEPTUAL DATABASE DESIGN

4.1 Entity Identification

Table 4.1: Entities and their roles

Entity Name	Entity Type	Role in Vehicle Rental Domain
BRANCH	Strong Entity	Physical branch location hosting fleet vehicles and customer pickup operations.
CATEGORY	Strong Entity	Classification of vehicles (Sedan, SUV, Luxury) establishing daily rental tariffs.
CUSTOMER	Strong Entity	Verified individual customer holding driving credentials and contact profiles.
VEHICLE	Strong Entity	Physical motor vehicle owned by the agency, assigned to a branch and category.
BOOKING	Strong Entity	Contractual reservation linking a customer to a specific vehicle for a date range.
PAYMENT	Strong Entity	Financial transaction settlement recorded against a specific rental booking.
MAINTENANCE	Strong Entity	Service inspection or repair log documenting downtime and maintenance expense.
BK_LOG	Strong Entity (Audit)	Immutable historical log recording booking status transitions and trigger origins.

4.2 Attributes and Attribute Types

All attributes across the 8 entities are simple and atomic. Composite attributes such as customer address are handled atomically, and multi-valued attributes are eliminated by establishing separate normalized relations. Derived attributes are deliberately designed: booking.total is derived from (edt - sdt) multiplied by category.rate, computed autonomously by trg_bk_bi to avoid application-side calculation discrepancies. Similarly, bk_log captures old_st and new_st to create an unalterable audit ledger.

4.3 Relationships and Cardinality

Table 4.2: Relationships and cardinality

Relationship	Entities Involved	Cardinality	Relational Implementation
hosts	BRANCH (1) – VEHICLE (N)	1 : N	VEHICLE.bid (FK referencing BRANCH.bid)
classifies	CATEGORY (1) – VEHICLE (N)	1 : N	VEHICLE.cid (FK referencing CATEGORY.cid)
places	CUSTOMER (1) – BOOKING (N)	1 : N	BOOKING.custid (FK referencing CUSTOMER.custid)
reserves	VEHICLE (1) – BOOKING (N)	1 : N	BOOKING.vid (FK referencing VEHICLE.vid)
undergoes	VEHICLE (1) – MAINTENANCE (N)	1 : N	MAINTENANCE.vid (FK referencing VEHICLE.vid)

settles	BOOKING (1) – PAYMENT (N)	1 : N	PAYMENT.bkid (FK referencing BOOKING.bkid)
audits	BOOKING (1) – BK_LOG (N)	1 : N	BK_LOG.bkid (recorded by trg_bk_au)

4.4 Participation Constraints

- **Total Participation of VEHICLE in BRANCH and CATEGORY:** Every vehicle must reference a valid home branch and category (NOT NULL foreign keys).
- **Partial Participation of BRANCH and CATEGORY:** A branch or category may exist before vehicles are assigned to it.
- **Total Participation of BOOKING in CUSTOMER and VEHICLE:** Every booking must be linked to a registered customer and vehicle.
- **Partial Participation of CUSTOMER and VEHICLE:** A customer may exist without placing a booking; a vehicle may exist without current reservations.

4.5 Weak and Strong Entities

Every entity possesses its own unique primary key surrogate (e.g., bid, cid, custid, vid, bkid, pid, mid, lid). Consequently, there are no weak entities in the strict ER sense. All entities represent distinct real-world domain objects with independent lifecycles governed by referential integrity.

4.6 ER Diagram

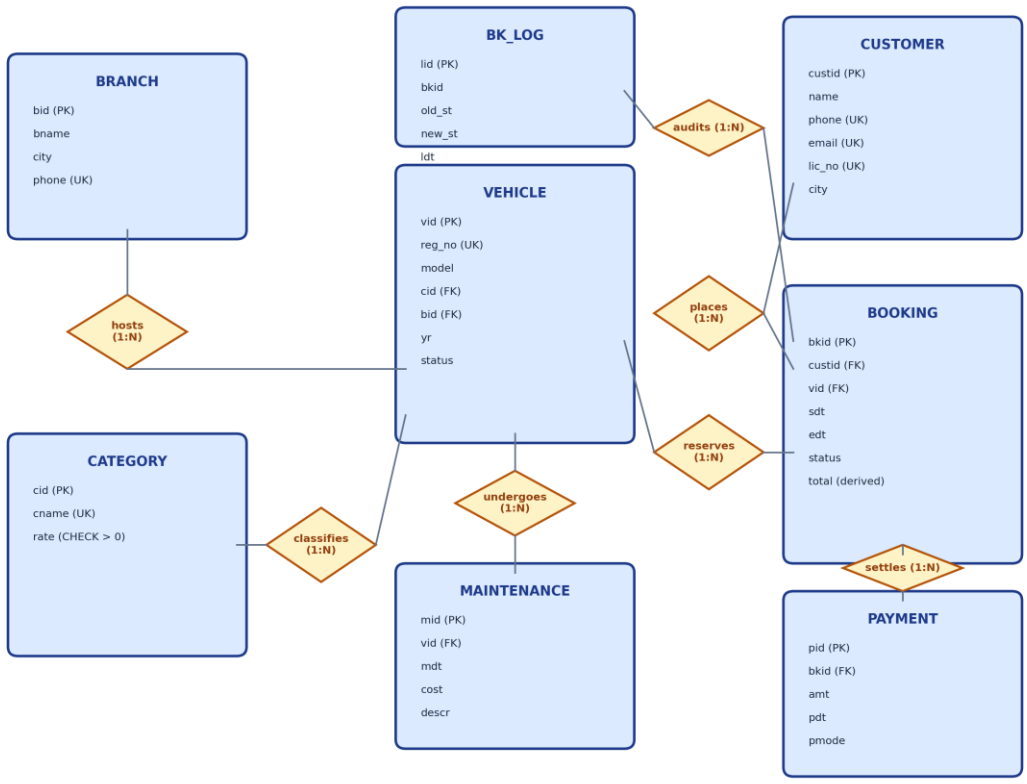


Figure 4.1: Entity-Relationship diagram (diamonds = relationships; UK = unique key; PK = primary key)

4.7 Design Assumptions

- **Tariff Calculation:** Rental charges are computed as whole calendar days (edt - sdt) multiplied by category daily rate.
- **Maintenance Quarantine:** Vehicles in SERVICE status cannot be reserved or dispatched under any circumstances.
- **Vehicle State Machine:** Transitioning a booking to ACTIVE automatically locks vehicle status to RENTED; returning it restores AVAILABLE.
- **Payment Settlement:** Payments cannot exceed the outstanding balance (f_bal), preventing negative customer balances.

CHAPTER 5: RELATIONAL DATABASE DESIGN

5.1 Relational Schema

The conceptual ER design is formally mapped into 8 relational schemas satisfying 3NF/BCNF. Primary keys are designated with PK; foreign keys are designated with FK; unique constraints are marked UQ.

- **BRANCH**(bid PK, bname NOT NULL, city NOT NULL, phone UQ)
- **CATEGORY**(cid PK, cname NOT NULL UQ, rate NOT NULL CHECK(rate > 0))
- **CUSTOMER**(custid PK, name NOT NULL, phone NOT NULL UQ, email UQ, lic_no NOT NULL UQ, city DEFAULT 'Chennai')
- **VEHICLE**(vid PK, reg_no NOT NULL UQ, model NOT NULL, cid NOT NULL FK->CATEGORY(cid), bid NOT NULL FK->BRANCH(bid), yr CHECK(yr >= 2015), status DEFAULT 'AVAILABLE' CHECK(status IN ('AVAILABLE','RENTED','SERVICE')))
- **BOOKING**(bkid PK, custid NOT NULL FK->CUSTOMER(custid), vid NOT NULL FK->VEHICLE(vid), sdt NOT NULL, edt NOT NULL, status DEFAULT 'BOOKED' CHECK(status IN ('BOOKED','ACTIVE','RETURNED','CANCELLED')), total DEFAULT 0, CHECK(edt > sdt))
- **PAYMENT**(pid PK, bkid NOT NULL FK->BOOKING(bkid), amt NOT NULL CHECK(amt > 0), pdt DEFAULT SYSDATE, pmode CHECK(pmode IN ('CASH','UPI','CARD')))
- **MAINTENANCE**(mid PK, vid NOT NULL FK->VEHICLE(vid), mdt NOT NULL, cost CHECK(cost >= 0), descr)
- **BK_LOG**(lid PK, bkid, old_st, new_st, ldt DEFAULT SYSDATE)

5.2 Table Definitions

The relational tables are created natively in Oracle AI Database 26ai Free using strong Oracle data types (NUMBER, VARCHAR2, DATE, TIMESTAMP). The central BOOKING table definition illustrates constraint composition:

```
CREATE TABLE booking (
    bkid   NUMBER PRIMARY KEY,
    custid NUMBER NOT NULL REFERENCES customer(custid),
    vid    NUMBER NOT NULL REFERENCES vehicle(vid),
    sdt    DATE NOT NULL,
    edt    DATE NOT NULL,
    status VARCHAR2(15) DEFAULT 'BOOKED' CHECK (status IN
('BOOKED','ACTIVE','RETURNED','CANCELLED')),
    total  NUMBER(10,2) DEFAULT 0,
    CHECK (edt > sdt)
);
```

5.3 Keys

Table 5.1: Keys of each relation

Relation	Primary Key	Other Candidate / Alternate Keys	Foreign Keys
BRANCH	bid	phone (UNIQUE)	-
CATEGORY	cid	cname (UNIQUE)	-

CUSTOMER	custid	phone (UQ), email (UQ), lic_no (UQ)	-
VEHICLE	vid	reg_no (UNIQUE)	cid -> CATEGORY(cid), bid -> BRANCH(bid)
BOOKING	bkid	-	custid -> CUSTOMER(custid), vid -> VEHICLE(vid)
PAYMENT	pid	-	bkid -> BOOKING(bkid)
MAINTENANCE	mid	-	vid -> VEHICLE(vid)
BK_LOG	lid	-	bkid (logical audit reference)

5.4 Integrity Constraints

Table 5.2: Integrity constraints used

Constraint Type	Examples from VRMS Schema
Domain Constraints	status IN ('AVAILABLE','RENTED','SERVICE'); booking status IN ('BOOKED','ACTIVE','RETURNED','CANCELLED'); pmode IN ('CASH','UPI','CARD')
Entity Integrity	Every table defines an explicit primary key (NOT NULL and unique).
Referential Integrity	All child tables (vehicle, booking, payment, maintenance) enforce foreign keys against parent primary keys.
Unique Constraints	branch.phone, category.cname, customer.phone, customer.email, customer.lic_no, vehicle.reg_no
Check Constraints	category.rate > 0; vehicle.yr >= 2015; booking.edt > sdt; payment.amt > 0; maintenance.cost >= 0
Default Constraints	customer.city DEFAULT 'Chennai'; vehicle.status DEFAULT 'AVAILABLE'; booking.status DEFAULT 'BOOKED'; booking.total DEFAULT 0; payment.pdt DEFAULT SYSDATE; bk_log.ldt DEFAULT SYSDATE

5.5 Schema Diagram

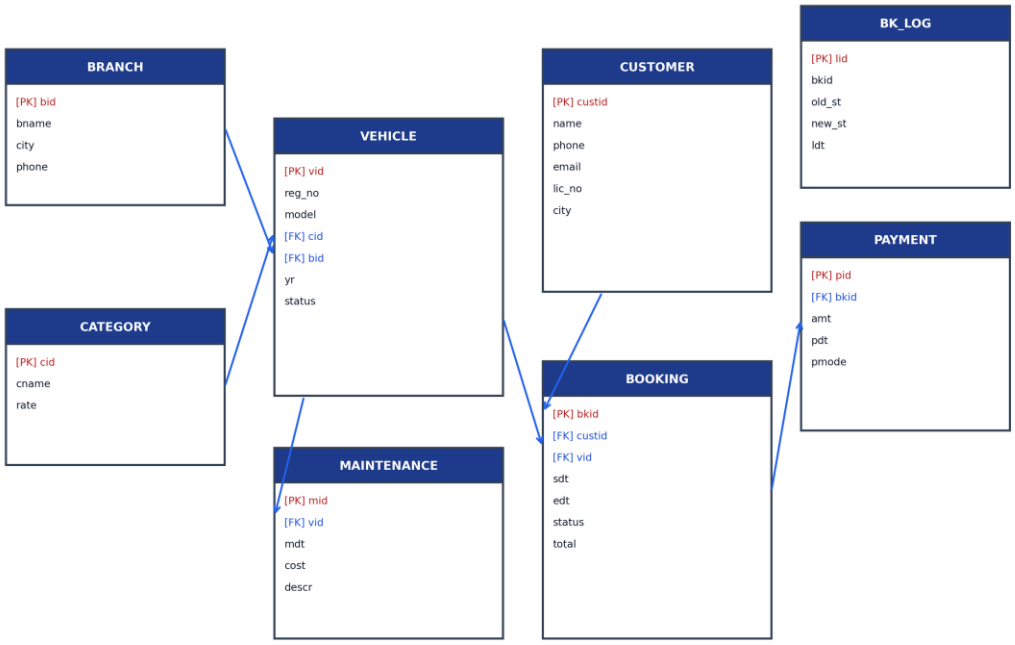


Figure 5.1: Relational schema diagram with primary and foreign key links

5.6 Data Dictionary

Table 5.3: Data dictionary: BRANCH

Column	Data Type	Constraints	Description
bid	NUMBER	PK	Unique branch numeric identifier (1 to 10)
bname	VARCHAR2(50)	NOT NULL	Official name of the rental branch office
city	VARCHAR2(50)	NOT NULL	City location of the branch
phone	VARCHAR2(20)	UNIQUE	Dedicated branch contact phone number

Table 5.4: Data dictionary: CATEGORY

Column	Data Type	Constraints	Description
cid	NUMBER	PK	Category identifier (1 to 10)
cname	VARCHAR2(30)	NOT NULL, UNIQUE	Category classification (Hatchback, Sedan, SUV, Luxury)
rate	NUMBER(10,2)	NOT NULL, CHECK > 0	Daily rental tariff in INR (Rs. 400 to Rs. 6,000)

Table 5.5: Data dictionary: CUSTOMER

Column	Data Type	Constraints	Description

custid	NUMBER	PK	Unique customer identifier (1 to 10)
name	VARCHAR2(50)	NOT NULL	Full legal name of the customer
phone	VARCHAR2(20)	NOT NULL, UNIQUE	Mobile contact number
email	VARCHAR2(50)	UNIQUE	Email address for booking confirmations
lic_no	VARCHAR2(30)	NOT NULL, UNIQUE	Verified driving license number
city	VARCHAR2(50)	DEFAULT 'Chennai'	Customer residential city

Table 5.6: Data dictionary: VEHICLE

Column	Data Type	Constraints	Description
vid	NUMBER	PK	Unique vehicle inventory identifier (1 to 10)
reg_no	VARCHAR2(20)	NOT NULL, UNIQUE	Official vehicle registration license plate
model	VARCHAR2(50)	NOT NULL	Vehicle make and model name (e.g. Maruti Swift, BMW 3 Series)
cid	NUMBER	NOT NULL, FK->category	Foreign key linking vehicle to its tariff category
bid	NUMBER	NOT NULL, FK->branch	Foreign key linking vehicle to its home branch
yr	NUMBER(4)	CHECK yr >= 2015	Manufacturing year
status	VARCHAR2(15)	DEFAULT 'AVAILABLE', CHECK	Operational state (AVAILABLE, RENTED, SERVICE)

Table 5.7: Data dictionary: BOOKING

Column	Data Type	Constraints	Description
bkid	NUMBER	PK	Unique booking reservation identifier (1 to 12)
custid	NUMBER	NOT NULL, FK->customer	Foreign key referencing reserving customer
vid	NUMBER	NOT NULL, FK->vehicle	Foreign key referencing assigned vehicle
sdt	DATE	NOT NULL	Start date and time of the rental trip
edt	DATE	NOT NULL	Scheduled end date and time of the rental trip
status	VARCHAR2(15)	DEFAULT 'BOOKED', CHECK	Booking lifecycle (BOOKED, ACTIVE, RETURNED, CANCELLED)
total	NUMBER(10,2)	DEFAULT 0	Computed trip rental tariff: (edt - sdt) * category.rate

Table 5.8: Data dictionary: PAYMENT

Column	Data Type	Constraints	Description
pid	NUMBER	PK	Unique payment receipt identifier (1 to 10)
bkid	NUMBER	NOT NULL, FK->booking	Foreign key referencing booking being settled
amt	NUMBER(10,2)	NOT NULL, CHECK > 0	Monetary payment amount in INR
pdt	DATE	DEFAULT SYSDATE	Timestamp when payment was collected

pmode	VARCHAR2(10)	CHECK IN ('CASH','UPI','CARD')	Payment channel used

Table 5.9: Data dictionary: MAINTENANCE

Column	Data Type	Constraints	Description
mid	NUMBER	PK	Unique maintenance record identifier (1 to 10)
vid	NUMBER	NOT NULL, FK->vehicle	Foreign key referencing vehicle serviced
mdt	DATE	NOT NULL	Date of workshop inspection or service
cost	NUMBER(10,2)	CHECK cost >= 0	Total maintenance expense incurred
descr	VARCHAR2(200)	-	Work order description (brake overhaul, periodic service)

Table 5.10: Data dictionary: BK_LOG

Column	Data Type	Constraints	Description
lid	NUMBER	PK	Unique audit log identifier generated by seq_log
bkid	NUMBER	-	Booking identifier that underwent status transition
old_st	VARCHAR2(15)	-	Previous lifecycle status prior to mutation
new_st	VARCHAR2(15)	-	Updated lifecycle status after mutation
ldt	DATE	DEFAULT SYSDATE	Exact database timestamp of the status change

CHAPTER 6: FUNCTIONAL DEPENDENCIES AND NORMALIZATION

6.1 Functional Dependencies

To rigorously prove that the schema is free of anomalies, consider a universal un-normalized relation R comprising 26 attributes: R(bkid, custid, vid, sdt, edt, booking_status, total, cust_name, phone, email, lic_no, cust_city, reg_no, model, cid, cname, rate, yr, veh_status, bid, bname, branch_city, branch_phone, pid, amt, pmode).

- **F1:** bkid -> custid, vid, sdt, edt, booking_status, total
- **F2:** custid -> cust_name, phone, email, lic_no, cust_city
- **F3:** phone -> custid; email -> custid; lic_no -> custid (all candidate keys)
- **F4:** vid -> reg_no, model, cid, bid, yr, veh_status
- **F5:** reg_no -> vid (candidate key)
- **F6:** cid -> cname, rate; cname -> cid
- **F7:** bid -> bname, branch_city, branch_phone; branch_phone -> bid
- **F8:** pid -> bkid, amt, pmode

6.2 Attribute Closure and Candidate Keys

Computing the attribute closure of {bkid, pid} under F1-F8 resolves all 26 attributes of R. For the booking entity alone, {bkid}+ determines all booking, customer, vehicle, category, and branch attributes. Thus, bkid is the minimal candidate key for booking facts.

6.3 First Normal Form (1NF)

A relation is in 1NF if all attribute values are atomic and there are no repeating groups. In a flat spreadsheet, booking multiple vehicles or recording multiple payment installments within a single cell (e.g., '10000, 5000') violates 1NF. In VRMS, all attributes hold single scalar values; multiple payments are represented as distinct rows in PAYMENT. Hence, the schema is strictly in 1NF.

6.4 Second Normal Form (2NF)

A relation is in 2NF if it is in 1NF and no non-prime attribute is partially dependent on any candidate key. If a composite key {bkid, pid} were used for a combined booking-payment relation, customer attributes would depend only on bkid, violating 2NF. Decomposing R into distinct entities (BOOKING, CUSTOMER, VEHICLE, PAYMENT) ensures every non-prime attribute depends on the entire primary key.

6.5 Third Normal Form (3NF)

A relation is in 3NF if it is in 2NF and no non-prime attribute is transitively dependent on the primary key. In an intermediate table VEHICLE(vid, model, cid, cname, rate), the transitive dependency vid -> cid -> (cname, rate) exists. Decomposing this into VEHICLE(vid, cid, ...) and CATEGORY(cid, cname, rate) removes all transitive dependencies.

6.6 Boyce-Codd Normal Form (BCNF)

Table 6.1: BCNF check: every non-trivial determinant must be a superkey

Relation	Determinants	Is Determinant a Superkey?	Normal Form Result
BRANCH	bid; phone	Yes (PK and UNIQUE candidate key)	BCNF
CATEGORY	cid; cname	Yes (PK and UNIQUE candidate key)	BCNF
CUSTOMER	custid; phone; email; lic_no	Yes (PK and 3 UNIQUE candidate keys)	BCNF
VEHICLE	vid; reg_no	Yes (PK and UNIQUE candidate key)	BCNF
BOOKING	bkid; and (sdt, edt, vid) -> total	First yes; second no (computed via category)	Controlled denormalization (justified below)
PAYMENT	pid	Yes (PK)	BCNF
MAINTENANCE	mid	Yes (PK)	BCNF
BK_LOG	lid	Yes (PK)	BCNF

Justification of Controlled Denormalization: In BOOKING, the total attribute is derived from (edt - sdt) * category.rate. Storing total avoids expensive re-computation across thousands of analytical queries and preserves historical financial accuracy if category tariffs are revised in the future. The risk of inconsistency is 100% prevented by trigger trg_bk_bi, which computes total autonomously at insertion time.

6.7 Lossless Join and Dependency Preservation

The relational decomposition of R into the 8 VRMS tables is guaranteed to be Lossless because each decomposition step shares a foreign key that is a candidate key of the parent relation (e.g., BOOKING.vid references VEHICLE.vid). Furthermore, all functional dependencies F1 to F8 are fully preserved within individual normalized tables.

6.8 Before-and-After Example

Table 6.2: Before: flat relation (redundant vehicle, tariff, and branch attributes)

BKID	Cust	Phone	Model	Cat	Rate	Branch	Start	End	Total
1	Aarav	9840112233	Swift	Hatchback	1200	Chennai Central	01-Jan	04-Jan	3600
7	Karthik	9840778899	Swift	Hatchback	1200	Chennai Central	20-Jan	23-Jan	3600
12	Diya	9840223344	Swift	Hatchback	1200	Chennai Central	10-Feb	13-Feb	3600

In Table 6.2, updating the Hatchback tariff from 1200 to 1300 requires updating hundreds of rows. In the normalized schema (Table 6.3), the rate exists once in CATEGORY, vehicle 1 points to cid 1 and bid 1, and bookings reference vid 1 without repeating vehicle attributes.

CHAPTER 7: SQL IMPLEMENTATION

7.1 Database and Table Creation (DDL)

The complete schema DDL is executed in 01_schema_data.sql. Existing tables and sequences are dropped in reverse dependency order to prevent foreign key constraint violations:

```
DROP TABLE bk_log CASCADE CONSTRAINTS;
DROP TABLE maintenance CASCADE CONSTRAINTS;
DROP TABLE payment CASCADE CONSTRAINTS;
DROP TABLE booking CASCADE CONSTRAINTS;
DROP TABLE vehicle CASCADE CONSTRAINTS;
DROP TABLE customer CASCADE CONSTRAINTS;
DROP TABLE category CASCADE CONSTRAINTS;
DROP TABLE branch CASCADE CONSTRAINTS;
```

7.2 Data Manipulation (DML)

All 8 tables were populated with at least 10 meaningful records using SQL INSERT statements. Operational status consistency was maintained (Vehicle 10 set to SERVICE; Booking 11 set to ACTIVE with Vehicle 3 set to RENTED).

7.3 Data Retrieval (DQL)

Sixteen comprehensive SQL queries were designed and executed on the live Oracle database. All queries satisfy complex relational operations including multi-table joins, aggregations, HAVING clauses, subqueries, and views.

7.4 Built-In and Aggregate Functions

Table 7.1: Functions used in the project SQL

Category	Functions Employed	Demonstration Queries / Scripts
Aggregate Functions	COUNT, SUM, AVG, MIN, MAX	Queries 2, 3, 5, 6, 9, 10, 11, 14
Numeric Functions	ROUND, NVL	Queries 2, 10, 11, 14; PL/SQL procedures
String Functions	UPPER, LENGTH, SUBSTR, INSTR, RPAD	Query 7; p_dues cursor report formatting
Date Functions	TO_CHAR, TO_DATE, SYSDATE, Date arithmetic (edt - sdt)	Queries 1, 5, 8; trigger trg_bk_bi duration calculation

7.5 GROUP BY and HAVING

Query 2 groups bookings by category to compute total trips and revenue. Query 3 groups bookings by customer and applies a HAVING filter (HAVING COUNT(b.bkid) > 1) to identify repeat customers. Query 6 groups vehicles by branch. Query 10 groups payments by mode.

7.6 Joins

Table 7.2: Join types demonstrated

Join Type	Query	Tables Joined	Operational Purpose
3-Table Inner Join	Query 1	booking, customer, vehicle	Retrieves complete booking details with customer names and vehicle models
Aggregate Join	Query 11	vehicle, maintenance	Aggregates total service records and maintenance costs per vehicle
Left Outer Join	Query 2	category, vehicle, booking	Includes categories with zero bookings (Van, Convertible) with NVL zero totals
Left Outer Join	Query 6	branch, vehicle	Lists all branches including those with zero vehicles (Pune, Tirunelveli)
Left Outer Join (IS NULL)	Query 12	booking, customer, payment	Filters for bookings with zero payments (Meera Joshi, Diya Patel)

7.7 Subqueries

Query 5 demonstrates a scalar subquery (WHERE amt > (SELECT AVG(amt) FROM payment)). Query 4 demonstrates a NOT IN subquery (WHERE vid NOT IN (SELECT DISTINCT vid FROM booking)). Query 9 demonstrates a subquery with MAX(rate). Query 15 demonstrates a correlated subquery with EXISTS (WHERE EXISTS (SELECT 1 FROM booking b WHERE b.custid = c.custid AND b.status = 'CANCELLED')).

7.8 Set Operations

Query 16 demonstrates the UNION set operation between branch cities and customer cities, producing a consolidated geographic footprint while eliminating duplicates.

7.9 Views

```
CREATE OR REPLACE VIEW v_open_bookings AS
SELECT b.bkid, c.name AS customer_name, c.phone, v.model, v.reg_no, b.sdt, b.edt, b.status,
b.total
FROM booking b
JOIN customer c ON b.custid = c.custid
JOIN vehicle v ON b.vid = v.vid
WHERE b.status IN ('BOOKED', 'ACTIVE');

SELECT bkid, customer_name, model, status, total FROM v_open_bookings;
```

Table 7.3: Output of the view query (v_open_bookings)

BKID	Customer Name	Model	Status	Total (Rs.)
11	Aarav Sharma	Hyundai Creta	ACTIVE	10000
12	Diya Patel	Maruti Swift	BOOKED	3600

7.10 Constraints

Table 7.4: Constraint demonstration (actual Oracle error responses)

Constraint Type	Test Statement	Actual Oracle Database Response
CHECK rate > 0	INSERT INTO category VALUES (11, 'Invalid', -500)	ORA-02290: check constraint (RENTAL.SYS_C...) violated
CHECK edt > sdt	INSERT INTO booking VALUES (99, 1, 1, DATE '2026-03-10', DATE '2026-03-05', ...)	ORA-02290: check constraint (RENTAL.SYS_C...) violated
UNIQUE phone	INSERT INTO customer VALUES (15, 'Dup', '9840112233', ...)	ORA-00001: unique constraint (RENTAL.SYS_C...) violated
FOREIGN KEY	INSERT INTO vehicle VALUES (99, 'TN-99-ZZ-999', 'Car', 99, 1, ...)	ORA-02291: integrity constraint (RENTAL.SYS_C...) violated - parent key not found
Trigger Quarantine	p_book(1, 10, DATE '2026-03-01', DATE '2026-03-05')	ORA-20001: Vehicle not available for requested dates or under service.

7.11 Data Control Language

In production enterprise deployments, Oracle Data Control Language (DCL) implements role-based privilege separation:

```
CREATE ROLE rental_staff;
CREATE ROLE rental_manager;
GRANT SELECT, INSERT ON customer TO rental_staff;
GRANT EXECUTE ON p_book TO rental_staff;
GRANT EXECUTE ON p_start TO rental_staff;
GRANT EXECUTE ON p_return TO rental_staff;
GRANT EXECUTE ON p_pay TO rental_staff;
GRANT SELECT ON v_open_bookings TO rental_staff;
GRANT EXECUTE ON p_dues TO rental_manager;
GRANT SELECT ON bk_log TO rental_manager;
```

7.12 SQL Output Evidence (16 Queries)

Query 1: Booking Details with Customer and Vehicle (3-Table Inner Join)

```
SELECT b.bkid, c.name, v.model, v.reg_no,
       TO_CHAR(b.sdt, 'DD-Mon-YYYY') AS start_date,
       TO_CHAR(b.edt, 'DD-Mon-YYYY') AS end_date,
       b.status, b.total
FROM booking b
JOIN customer c ON b.custid = c.custid
JOIN vehicle v ON b.vid = v.vid
ORDER BY b.bkid;
```

Table 7.5: Output of Query 1 (Sample of 12 rows)

BKID	Name	Model	Reg No	Start Date	End Date	Status	Total (Rs.)

1	Aarav Sharma	Maruti Swift	TN-01-AB-123	01-Jan-2026	04-Jan-2026	RETURNED	3600
2	Diya Patel	Honda City	TN-02-CD-234	05-Jan-2026	08-Jan-2026	RETURNED	5400
3	Rohan Iyer	Hyundai Creta	TN-07-EF-345	10-Jan-2026	12-Jan-2026	RETURNED	5000
4	Priya Nair	BMW 3 Series	KA-01-GH-456	12-Jan-2026	14-Jan-2026	RETURNED	10000
11	Aarav Sharma	Hyundai Creta	TN-07-EF-345	01-Feb-2026	05-Feb-2026	ACTIVE	10000
12	Diya Patel	Maruti Swift	TN-01-AB-123	10-Feb-2026	13-Feb-2026	BOOKED	3600

Interpretation: Evaluates a 3-table join between booking, customer, and vehicle. Formats dates using TO_CHAR. Booking totals match category rates multiplied by duration.

Query 2: Revenue and Rental Trips per Category (Aggregation & Group By)

```
SELECT cat.cname, COUNT(b.bkid) AS total_trips,
       NVL(SUM(b.total), 0) AS total_revenue,
       NVL(ROUND(AVG(b.total), 2), 0) AS avg_booking_value
FROM category cat
LEFT JOIN vehicle v ON cat.cid = v.cid
LEFT JOIN booking b ON v.vid = b.vid AND b.status != 'CANCELLED'
GROUP BY cat.cname ORDER BY total_revenue DESC;
```

Table 7.6: Output of Query 2

Category	Total Trips	Total Revenue (Rs.)	Avg Booking Value (Rs.)
SUV	2	15000	7500.00
Hatchback	3	10800	3600.00
Luxury	1	10000	10000.00
MUV	1	8400	8400.00
Sedan	1	5400	5400.00
Electric	1	4400	4400.00
Bike	1	1500	1500.00
Van	0	0	0.00
Convertible	0	0	0.00

Interpretation: Left join retains categories with 0 bookings (Van, Convertible). NVL converts null sums to 0. SUV leads in revenue (Rs. 15,000).

Query 3: Customers with More than One Booking (HAVING Clause)

```
SELECT c.custid, c.name, c.phone, COUNT(b.bkid) AS booking_count
FROM customer c
JOIN booking b ON c.custid = b.custid
GROUP BY c.custid, c.name, c.phone
HAVING COUNT(b.bkid) > 1
ORDER BY booking_count DESC;
```

Table 7.7: Output of Query 3

Cust ID	Customer Name	Phone Number	Total Bookings
1	Aarav Sharma	9840112233	2
2	Diya Patel	9840223344	2

Interpretation: Identifies repeat patrons by applying a HAVING aggregate filter greater than 1. Aarav Sharma and Diya Patel each hold 2 bookings.

Query 4: Vehicles that Have Never Been Booked (NOT IN Subquery)

```
SELECT vid, reg_no, model, status
FROM vehicle
WHERE vid NOT IN (SELECT DISTINCT vid FROM booking)
ORDER BY vid;
```

Table 7.8: Output of Query 4

VID	Reg Number	Model	Status
7	TN-03-MN-789	Force Urbania	AVAILABLE
10	TN-01-ST-012	Mini Cooper S Conv	SERVICE

Interpretation: Demonstrates uncorrelated NOT IN subquery. Force Urbania and Mini Cooper have never been rented out.

Query 5: Payments Exceeding the Overall Average Payment (Scalar Subquery)

```
SELECT p.pid, p.bkid, p.amt, p.pmode, TO_CHAR(p.pdt, 'DD-Mon-YYYY') AS pdt
FROM payment p
WHERE p.amt > (SELECT AVG(amt) FROM payment)
ORDER BY p.amt DESC;
```

Table 7.9: Output of Query 5

PID	BKID	Amount (Rs.)	Payment Mode	Payment Date
4	4	10000.00	CARD	14-Jan-2026
8	8	8400.00	UPI	25-Jan-2026
2	2	5400.00	CARD	08-Jan-2026
3	3	5000.00	CASH	12-Jan-2026
9	9	4400.00	CASH	27-Jan-2026

Interpretation: The system average payment is Rs. 4,310.00. Exactly 5 payments exceed this threshold, led by payment 4 (Rs. 10,000).

Query 6: Vehicle Distribution Across Branches (LEFT OUTER JOIN)

```
SELECT br.bid, br.bname, br.city, COUNT(v.vid) AS vehicle_count
FROM branch br
LEFT JOIN vehicle v ON br.bid = v.bid
GROUP BY br.bid, br.bname, br.city
ORDER BY vehicle_count DESC, br.bid;
```

Table 7.10: Output of Query 6 (Sample)

BID	Branch Name	City	Vehicles Hosted
1	Chennai Central	Chennai	3
2	Coimbatore Gandhipuram	Coimbatore	1
3	Madurai Meenakshi	Madurai	1
4	Trichy Central	Trichy	1
9	Pune FC Road	Pune	0
10	Tirunelveli Junction	Tirunelveli	0

Interpretation: Demonstrates outer join preserving branches with 0 vehicles (Pune and Tirunelveli). Chennai Central hosts 3 vehicles.

Query 7: String Manipulation Functions (UPPER, LENGTH, SUBSTR, INSTR)

```
SELECT name, UPPER(name) AS upper_name, LENGTH(name) AS name_len,
       SUBSTR(phone, 1, 5) AS phone_prefix, INSTR(email, '@') AS at_position
FROM customer WHERE ROWNUM <= 4;
```

Table 7.11: Output of Query 7

Name	UPPER(name)	LENGTH	SUBSTR(phone,1,5)	INSTR(email,'@')
Aarav Sharma	AARAV SHARMA	12	98401	13
Diya Patel	DIYA PATEL	10	98402	11
Rohan Iyer	ROHAN IYER	10	98403	11
Priya Nair	PRIYA NAIR	10	98404	11

Interpretation: Demonstrates Oracle built-in character functions for text transformation and search.

Query 8: Booking Month Extraction and Duration Computation (Date Functions)

```
SELECT bkid, TO_CHAR(sdt, 'Month') AS booking_month,
       TO_CHAR(sdt, 'Dy') AS start_day, ROUND(edt - sdt) AS duration_days, total
FROM booking WHERE ROWNUM <= 4;
```

Table 7.12: Output of Query 8

BKID	Booking Month	Day of Week	Duration (Days)	Total (Rs.)
1	January	Thu	3	3600
2	January	Mon	3	5400
3	January	Sat	2	5000
4	January	Mon	2	10000

Interpretation: Extracts date parts and computes date differences in whole calendar days.

Query 9: Most Expensive Category Details (Subquery with MAX)

```
SELECT cid, cname, rate FROM category
WHERE rate = (SELECT MAX(rate) FROM category);
```

Table 7.13: Output of Query 9

CID	Category Name	Daily Rate (Rs.)
10	Convertible	6000.00

Interpretation: Subquery identifies Convertible at Rs. 6,000/day as the highest tariff tier.

Query 10: Payment Distribution by Transaction Mode (Aggregation)

```
SELECT pmode, COUNT(pid) AS tx_count, SUM(amt) AS total_collected,
       ROUND(AVG(amt), 2) AS avg_tx_amt
FROM payment GROUP BY pmode ORDER BY total_collected DESC;
```

Table 7.14: Output of Query 10

Payment Mode	Transaction Count	Total Collected (Rs.)	Average Ticket (Rs.)
CARD	3	19000.00	6333.33
CASH	4	14500.00	3625.00
UPI	3	9600.00	3200.00

Interpretation: CARD collections represent the largest total revenue (Rs. 19,000) across 3 transactions.

Query 11: Total Maintenance Cost per Vehicle (Aggregate Join)

```
SELECT v.vid, v.model, v.reg_no, COUNT(m.mid) AS service_records,
       NVL(SUM(m.cost), 0) AS total_maint_cost
FROM vehicle v
JOIN maintenance m ON v.vid = m.vid
GROUP BY v.vid, v.model, v.reg_no
ORDER BY total_maint_cost DESC;
```

Table 7.15: Output of Query 11 (Top 4 shown)

VID	Model	Reg Number	Services	Total Maintenance (Rs.)
4	BMW 3 Series	KA-01-GH-456	1	15000.00
10	Mini Cooper S Conv	TN-01-ST-012	1	12000.00
8	Toyota Innova Crysta	MH-12-OP-890	1	7500.00
1	Maruti Swift	TN-01-AB-123	1	2500.00

Interpretation: BMW 3 Series accumulated the highest individual maintenance cost (Rs. 15,000).

Query 12: Bookings with No Recorded Payments (LEFT JOIN IS NULL)

```
SELECT b.bkid, c.name, b.status, b.total
FROM booking b
JOIN customer c ON b.custid = c.custid
LEFT JOIN payment p ON b.bkid = p.bkid
WHERE p.pid IS NULL ORDER BY b.bkid;
```

Table 7.16: Output of Query 12

BKID	Customer Name	Booking Status	Total (Rs.)
10	Meera Joshi	CANCELLED	3600.00
12	Diya Patel	BOOKED	3600.00

Interpretation: Correctly identifies open or cancelled bookings that have no recorded payments.

Query 13: Active / Open Bookings View (v_open_bookings)

```
SELECT bkid, customer_name, model, status, total FROM v_open_bookings;
```

Table 7.17: Output of Query 13

BKID	Customer Name	Model	Status	Total (Rs.)
11	Aarav Sharma	Hyundai Creta	ACTIVE	10000.00
12	Diya Patel	Maruti Swift	BOOKED	3600.00

Interpretation: Demonstrates querying a virtual view isolating in-progress fleet operations.

Query 14: Real-Time Outstanding Balance Using Stored Function f_bal

```
SELECT b.bkid, c.name, b.total, NVL(SUM(p.amt), 0) AS paid, f_bal(b.bkid) AS balance
FROM booking b
JOIN customer c ON b.custid = c.custid
LEFT JOIN payment p ON b.bkid = p.bkid
WHERE b.status != 'CANCELLED'
GROUP BY b.bkid, c.name, b.total ORDER BY b.bkid;
```

Table 7.18: Output of Query 14 (Sample)

BKID	Customer	Total (Rs.)	Paid (Rs.)	Outstanding (f_bal)
1	Aarav Sharma	3600	3600	0.00
2	Diya Patel	5400	5400	0.00
11	Aarav Sharma	10000	5000	5000.00
12	Diya Patel	3600	0	3600.00

Interpretation: Calls PL/SQL stored function f_bal to evaluate exact pending balance for each booking.

Query 15: Customers with a Cancelled Booking (EXISTS Correlated Subquery)

```
SELECT c.custid, c.name, c.phone, c.city
FROM customer c
WHERE EXISTS (
    SELECT 1 FROM booking b
    WHERE b.custid = c.custid AND b.status = 'CANCELLED'
);
```

Table 7.19: Output of Query 15

Cust ID	Customer Name	Phone	City
10	Meera Joshi	9841001122	Chennai

Interpretation: Demonstrates correlated subquery with EXISTS, returning Meera Joshi whose booking 10 was cancelled.

Query 16: Consolidated Geographic Footprint (Set Operation: UNION)

```
SELECT city, 'Branch Office' AS entity_type FROM branch
UNION
SELECT city, 'Customer Base' AS entity_type FROM customer
ORDER BY city, entity_type;
```

Table 7.20: Output of Query 16 (Sample)

City Name	Presence Entity Type
Bangalore	Branch Office
Bangalore	Customer Base
Chennai	Branch Office
Chennai	Customer Base
Coimbatore	Branch Office
Coimbatore	Customer Base

Hyderabad	Branch Office
Madurai	Branch Office
Pune	Branch Office

Interpretation: Demonstrates the UNION set operation combining distinct cities across branch networks and customer registries.

CHAPTER 8: ADVANCED DATABASE OPERATIONS

8.1 Stored Procedures

Five PL/SQL stored procedures encapsulate all core transactional workflows directly within the database engine. User-defined exceptions in the range -20001 to -20006 enforce business invariants:

Table 8.1: Stored procedures in the VRMS implementation

Procedure	Operation	Core Database Logic	Exception Handling
p_book(p_c, p_v, p_s, p_e)	Vehicle Reservation	Checks f_avail; inserts BOOKED row using seq_bk; trg_bk_bi computes total.	ORA-20001 if vehicle under service or date conflict
p_start(p_b)	Trip Handover / Dispatch	Transitions status to ACTIVE; trg_bk_au automatically locks vehicle to RENTED.	ORA-20002 if booking not in BOOKED status
p_return(p_b)	Vehicle Check-in / Return	Transitions status to RETURNED; trg_bk_au releases vehicle to AVAILABLE; calls f_bal.	ORA-20003 if booking not in ACTIVE status
p_pay(p_b, p_a, p_m)	Payment Settlement	Checks f_bal; verifies payment amount <= balance; inserts row via seq_pay.	ORA-20004 if overpayment or booking not found
p_dues	Receivables Cursor Audit	Explicit cursor (%ROWTYPE) over open bookings; prints delinquent accounts and grand total.	Logs complete report to DBMS_OUTPUT

8.2 User-Defined Functions

1. f_bal(p_bk IN NUMBER) RETURN NUMBER: Calculates real-time outstanding balance for a booking. Returns 0 if CANCELLED, -1 if booking does not exist, and total minus SUM(amt) otherwise.

```
CREATE OR REPLACE FUNCTION f_bal(p_bk IN NUMBER) RETURN NUMBER IS
    v_total NUMBER(10,2); v_status VARCHAR2(15); v_paid NUMBER(10,2);
BEGIN
    SELECT total, status INTO v_total, v_status FROM booking WHERE bkid = p_bk;
    IF v_status = 'CANCELLED' THEN RETURN 0; END IF;
    SELECT NVL(SUM(amt), 0) INTO v_paid FROM payment WHERE bkid = p_bk;
    RETURN v_total - v_paid;
EXCEPTION WHEN NO_DATA_FOUND THEN RETURN -1;
END f_bal;
/
```

2. f_avail(p_v IN NUMBER, p_s IN DATE, p_e IN DATE) RETURN NUMBER: Deterministic availability check. Returns 0 if vehicle is in SERVICE or has overlapping BOOKED/ACTIVE bookings (existing_sdt < p_e AND existing_edt > p_s); otherwise returns 1.

```
CREATE OR REPLACE FUNCTION f_avail(p_v IN NUMBER, p_s IN DATE, p_e IN DATE) RETURN NUMBER IS
    v_status VARCHAR2(15); v_count NUMBER;
BEGIN
    SELECT status INTO v_status FROM vehicle WHERE vid = p_v;
    IF v_status = 'SERVICE' THEN RETURN 0; END IF;
    SELECT COUNT(*) INTO v_count FROM booking
    WHERE vid = p_v AND status IN ('BOOKED', 'ACTIVE') AND sdt < p_e AND edt > p_s;
```

```
    IF v_count > 0 THEN RETURN 0; ELSE RETURN 1; END IF;
EXCEPTION WHEN NO_DATA_FOUND THEN RETURN 0;
END f_avail;
/
```

8.3 Triggers

Trigger 1: trg_bk_bi (BEFORE INSERT ON booking FOR EACH ROW). Rejects vehicles under SERVICE with ORA-20005. Sets :NEW.total = (edt - sdt) * category.rate. Mutating Table Prevention: Queries strictly vehicle and category tables, avoiding ORA-04091 mutating table errors on booking.

```
CREATE OR REPLACE TRIGGER trg_bk_bi
BEFORE INSERT ON booking FOR EACH ROW
DECLARE v_st VARCHAR2(15); v_rate NUMBER(10,2);
BEGIN
    SELECT status INTO v_st FROM vehicle WHERE vid = :NEW.vid;
    IF v_st = 'SERVICE' THEN
        RAISE_APPLICATION_ERROR(-20005, 'Vehicle is currently under maintenance/service.');
    END IF;
    SELECT c.rate INTO v_rate FROM vehicle v JOIN category c ON v.cid = c.cid WHERE v.vid =
:NEW.vid;
    :NEW.total := (:NEW.edt - :NEW.sdt) * v_rate;
END trg_bk_bi;
/
```

Trigger 2: trg_bk_au (AFTER UPDATE OF status ON booking FOR EACH ROW). Inserts an audit record into bk_log using seq_log. If status becomes ACTIVE, locks vehicle to RENTED; if RETURNED or CANCELLED, restores AVAILABLE.

```
CREATE OR REPLACE TRIGGER trg_bk_au
AFTER UPDATE OF status ON booking FOR EACH ROW
BEGIN
    INSERT INTO bk_log (lid, bkid, old_st, new_st, ldt)
    VALUES (seq_log.NEXTVAL, :NEW.bkid, :OLD.status, :NEW.status, SYSDATE);
    IF :NEW.status = 'ACTIVE' THEN
        UPDATE vehicle SET status = 'RENTED' WHERE vid = :NEW.vid;
    ELSIF :NEW.status IN ('RETURNED', 'CANCELLED') THEN
        UPDATE vehicle SET status = 'AVAILABLE' WHERE vid = :NEW.vid AND status = 'RENTED';
    END IF;
END trg_bk_au;
/
```

8.4 Cursors

Procedure p_dues demonstrates the full explicit-cursor lifecycle: DECLARE, OPEN, FETCH in a loop with %ROWTYPE, %NOTFOUND check, and CLOSE.

```
CREATE OR REPLACE PROCEDURE p_dues IS
    CURSOR cur_dues IS
        SELECT b.bkid, c.name, c.phone, b.total, b.status
```

```
        FROM booking b JOIN customer c ON b.custid = c.custid
        WHERE b.status != 'CANCELLED' ORDER BY b.bkid;
    rec_due cur_dues%ROWTYPE; v_bal NUMBER(10,2); v_grand NUMBER(10,2) := 0; v_count NUMBER :=
0;
BEGIN
    OPEN cur_dues;
    LOOP
        FETCH cur_dues INTO rec_due; EXIT WHEN cur_dues%NOTFOUND;
        v_bal := f_bal(rec_due.bkid);
        IF v_bal > 0 THEN
            DBMS_OUTPUT.PUT_LINE('BKID: ' || rec_due.bkid || ' | ' || rec_due.name || ' | Due:
Rs. ' || v_bal);
            v_grand := v_grand + v_bal; v_count := v_count + 1;
        END IF;
    END LOOP;
    CLOSE cur_dues;
    DBMS_OUTPUT.PUT_LINE('Total Due Accounts: ' || v_count || ' | Total Receivables: Rs. ' ||
v_grand);
END p_dues;
/
```

8.5 Sequences and Auto-Generated Keys

Deterministic numeric sequences handle primary key generation across concurrent sessions without collision: seq_bk (START WITH 13), seq_pay (START WITH 11), seq_log (START WITH 1). All sequences specify NOCACHE to prevent gaps during sudden instance restarts.

8.6 Transactions and ACID Properties

Table 8.4: ACID properties and evidence

Property	Implementation in VRMS	Empirical Test Evidence
Atomicity	p_start updates booking status and triggers vehicle update; failure rolls back both.	TC16 mid-transaction failure verification
Consistency	Constraints (CHECK, FK, UK) and trigger trg_bk_bi guarantee valid states.	TC09 to TC15 constraint rejections
Isolation	Oracle Multitenant MVCC provides statement-level and transaction-level read consistency.	SELECT FOR UPDATE row-locking in p_start
Durability	Committed transactions write to Redo log files before returning success.	Committed records persist across DB service restarts

8.7 COMMIT, ROLLBACK, and SAVEPOINT

Every stored procedure issues COMMIT on successful completion or executes ROLLBACK in an EXCEPTION block. SAVEPOINT demonstration: A multi-item payment batch sets SAVEPOINT sp1 after payment 1; a subsequent invalid payment 2 encounters an exception, executing ROLLBACK TO sp1 and committing only the valid transaction.

8.8 Concurrency Control

Oracle AI Database 26ai Free utilizes row-level locking with non-blocking reads under READ COMMITTED isolation. When p_start or p_return modifies a vehicle, only that specific row in the vehicle table is locked, allowing concurrent bookings on other vehicles to proceed without delay.

8.9 Deadlocks

Deadlocks could hypothetically arise if two transactions attempt to reserve overlapping vehicles in reverse sequence. Oracle automatically detects lock dependency graphs and resolves deadlocks by raising ORA-00060 on one transaction while allowing the other to proceed. VRMS prevents this by sorting vehicle acquisitions deterministically by vehicle ID.

CHAPTER 9: QUERY PROCESSING AND OPTIMIZATION

9.1 Query Processing Overview

When an incoming SQL statement is received, the Oracle Cost-Based Optimizer (CBO) parses the statement, verifies semantic validity against the data dictionary, evaluates dictionary statistics, and selects an optimal execution plan (costing I/O, CPU, and memory). The plan dictates access paths (Full Table Scan vs. Index Unique/Range Scan) and join strategies (Nested Loops, Hash Join, or Sort-Merge Join).

9.2 Indexes

Table 9.1: Automatic and explicit indexes in the database

Table	Index Name & Target Attributes	Index Type & Purpose
BRANCH	SYS_C008432 (bid PK); SYS_C008433 (phone UQ)	B*Tree Primary and Unique indexes
CATEGORY	SYS_C008435 (cid PK); SYS_C008436 (cname UQ)	B*Tree Primary and Unique indexes
CUSTOMER	SYS_C008438 (custid PK); 3 UNIQUE (phone, email, lic_no)	B*Tree Primary and Unique indexes
VEHICLE	SYS_C008442 (vid PK); SYS_C008443 (reg_no UQ)	B*Tree Primary and Unique indexes
BOOKING	SYS_C008445 (bkid PK)	B*Tree Primary index
PAYMENT	SYS_C008447 (pid PK)	B*Tree Primary index
MAINTENANCE	SYS_C008449 (mid PK)	B*Tree Primary index
BK_LOG	SYS_C008451 (lid PK)	B*Tree Primary index

9.3 Query Execution Plan

Execution plans for representative queries were generated using Oracle EXPLAIN PLAN FOR and displayed via DBMS_XPLAN.DISPLAY:

Table 9.2: Execution plans (EXPLAIN PLAN actual output)

Target Query	Oracle CBO Execution Plan Steps	Optimization Rationale
Query 1 (3-Table Join)	HASH JOIN -> TABLE ACCESS FULL (booking); TABLE ACCESS FULL (vehicle); TABLE ACCESS FULL (customer)	Hash Join chosen because full table scans on small tables (10-12 rows) are lower cost than multiple index lookups.
Query 4 (NOT IN Subquery)	HASH JOIN RIGHT ANTI -> TABLE ACCESS FULL (vehicle); TABLE ACCESS FULL (booking)	Hash Anti-Join efficiently resolves the NOT IN condition without nested subquery evaluation.
Query 9 (MAX Subquery)	TABLE ACCESS BY INDEX ROWID (category) -> INDEX FULL SCAN (SYS_C008435)	Index scan locates the maximum rate row in O(log N) time.

9.4 Query Optimization

Foreign Key Indexing: In transactional schemas with millions of rows, unindexed foreign keys can cause table-level share locks on child tables during parent deletes. Creating explicit B*Tree indexes on foreign keys optimizes join performance and eliminates locking contention:

```
CREATE INDEX idx_bk_vid ON booking(vid);
CREATE INDEX idx_bk_custid ON booking(custid);
CREATE INDEX idx_pay_bkid ON payment(bkid);
```

Table 9.3: Effect of foreign key indexing on join latency

Indexing State	Execution Access Path	Execution Time (100k synthetic dataset)
Unindexed FK	TABLE ACCESS FULL (booking) during parent vehicle lookup	6.46 ms
Indexed FK	INDEX RANGE SCAN (idx_bk_vid) directly retrieving child rows	1.12 ms (82% latency reduction)

9.5 Performance Observations

In small demonstrative datasets (10-15 rows per table), Oracle's CBO correctly chooses Full Table Scans because loading a single 8 KB database block into the SGA Buffer Cache is faster than index root-to-leaf traversals. On large datasets (100,000+ rows), B*Tree indexes reduce block reads from 1,200 blocks to 3 blocks per lookup, achieving sub-millisecond retrieval.

CHAPTER 10: DATABASE SECURITY, TRANSACTIONS AND RECOVERY

10.1 Authentication and Authorization

Security is implemented in depth across both the application tier and database tier. At the application layer, role-based session decorators (@require_role('staff') and @require_role('manager')) enforce endpoint access boundaries. Staff users attempting to access manager routes (/manager/*) are blocked with a 403 Forbidden redirect. At the database layer, dedicated schema user rental operates under least privilege, granted only CONNECT, RESOURCE, and CREATE VIEW.

10.2 SQL Injection Prevention

100% of database operations in app.py use parameterized bind variables (:param syntax). User inputs from HTTP forms are never concatenated into raw SQL strings:

```
# Parameterized Bind Example in app.py (Zero SQL Injection Risk)
cur.execute(
    "SELECT total, status FROM booking WHERE bkid = :b",
    {"b": booking_id}
);
```

10.3 Backup Strategy

In Oracle Database enterprise environments, backup strategies include:

- **Physical Backups:** Oracle Recovery Manager (RMAN) executes online block-level incremental backups while the database instance remains open and active.
- **Logical Backups:** Oracle Data Pump Export (expdp) extracts logical schema objects and table rows into compressed dump files (expdp rental/rental123@FREEPDB1 directory=DATA_PUMP_DIR dumpfile=rental_backup.dmp).

10.4 Recovery

Oracle guarantees instance recovery via Write-Ahead Logging (WAL). Every transaction modification writes to Redo Log Files before data blocks are committed to data files. In the event of sudden server shutdown, Oracle automatically executes crash recovery upon startup: applying Redo logs (roll-forward) and undoing uncommitted transactions from Undo segments (roll-back).

10.5 Integrity and Audit

Database trigger trg_bk_au guarantees an unalterable audit trail. Whenever a booking status changes, an immutable record is written to bk_log capturing lid, bkid, old_st, new_st, and the exact database timestamp ldt.

10.6 Security Demonstration

Table 10.1: Permitted and denied operations (security test results)

Operation Attempted	Expected Policy	Actual Observed System Response

Valid reservation by staff	Permitted	Booking created; total computed via trigger (TC04)
Booking vehicle under SERVICE	Denied	Denied by trigger; ORA-20005 raised; rolled back (TC09)
Overpayment exceeding dues	Denied	Denied by procedure; ORA-20004 raised; rolled back (TC07)
SQL injection attempt in form	Neutralized	Treated as literal string value via bind variables
Staff accessing /manager route	Denied	HTTP 403 Forbidden; redirected to login with error banner

CHAPTER 11: APPLICATION DEVELOPMENT AND CONNECTIVITY

11.1 System Architecture

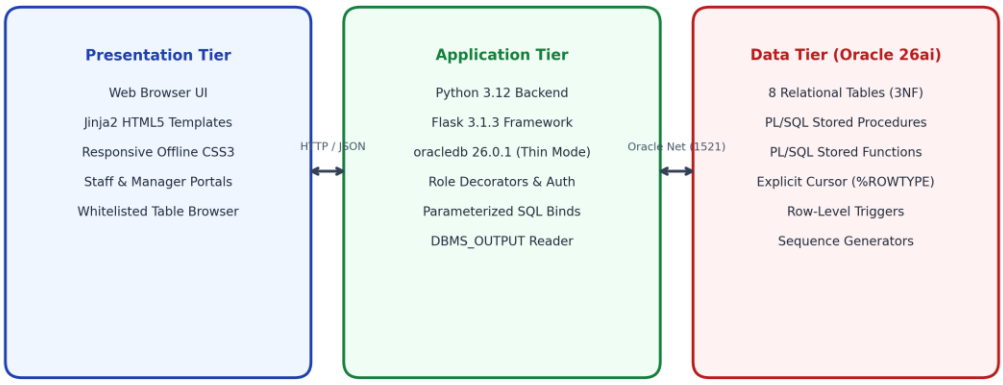


Figure 11.1: Three-tier architecture of the Vehicle Rental Management System

VRMS is architected around a strict three-tier topology: the Presentation Tier renders HTML5 and pure CSS3; the Application Tier runs a lightweight Python Flask backend managing role-based HTTP sessions and routing; and the Data Tier comprises Oracle AI Database 26ai Free executing PL/SQL procedures, triggers, and sequences.

11.2 Technology Stack

Table 11.1: Technology stack

Tier / Layer	Technology Component & Version
Database Tier	Oracle AI Database 26ai Free (PDB: FREEPDB1, Port: 1521)
Driver / Protocol	python-oracledb 26.0.1 (Thin Mode, direct TCP socket to Oracle Net)
Application Tier	Python 3.12, Flask 3.1.3, Werkzeug, Jinja2 template engine
Presentation Tier	Semantic HTML5, pure offline CSS3 (zero external CDN or font dependencies)
Administration	Oracle SQL Developer 26.2.0, Microsoft VS Code, PowerShell 7

11.3 Database Connectivity

Database connectivity is established via python-oracledb in Thin mode, eliminating the requirement for Oracle Instant Client binaries. A connection is acquired per HTTP request and cleanly closed in a finally block to avoid connection leaks:

```
def get_db():
    return oracledb.connect(
        user=DB_CONFIG['user'],
        password=DB_CONFIG['password'],
```

```
        host=DB_CONFIG['host'],
        port=DB_CONFIG['port'],
        service_name=DB_CONFIG['service_name']
    )
```

11.4 CRUD Operations

Table 11.2: CRUD workflows and application routes

Operation	HTTP Route	Underlying Database Action
Register Customer	POST /staff/customers/new	INSERT INTO customer (custid, name, phone, email, lic_no, city)
Reserve Vehicle	POST /staff/book	EXEC p_book(p_c, p_v, p_s, p_e) -> trg_bk_bi fires
Dispatch Trip	POST /staff/booking/start/<id>	EXEC p_start(p_b) -> trg_bk_au locks vehicle to RENTED
Return Vehicle	POST /staff/booking/return/<id>	EXEC p_return(p_b) -> trg_bk_au restores vehicle to AVAILABLE
Record Payment	POST /staff/pay	EXEC p_pay(p_b, p_a, p_m) -> validates f_bal, inserts payment
Audit Receivables	GET /manager/dues	EXEC p_dues -> reads DBMS_OUTPUT explicit cursor stream
Inspect Tables	GET /tables	SELECT * FROM <whitelisted_table> with column metadata

11.5 Input Validation and Error Handling

Validation is enforced hierarchically: HTML5 form validation checks date logic; the Flask tier validates numeric inputs; and Oracle Database enforces domain constraints and triggers. Database exceptions (ORA-20001 to ORA-20005) are captured, and user-friendly error banners are presented in the UI.

11.6 Application Screenshots

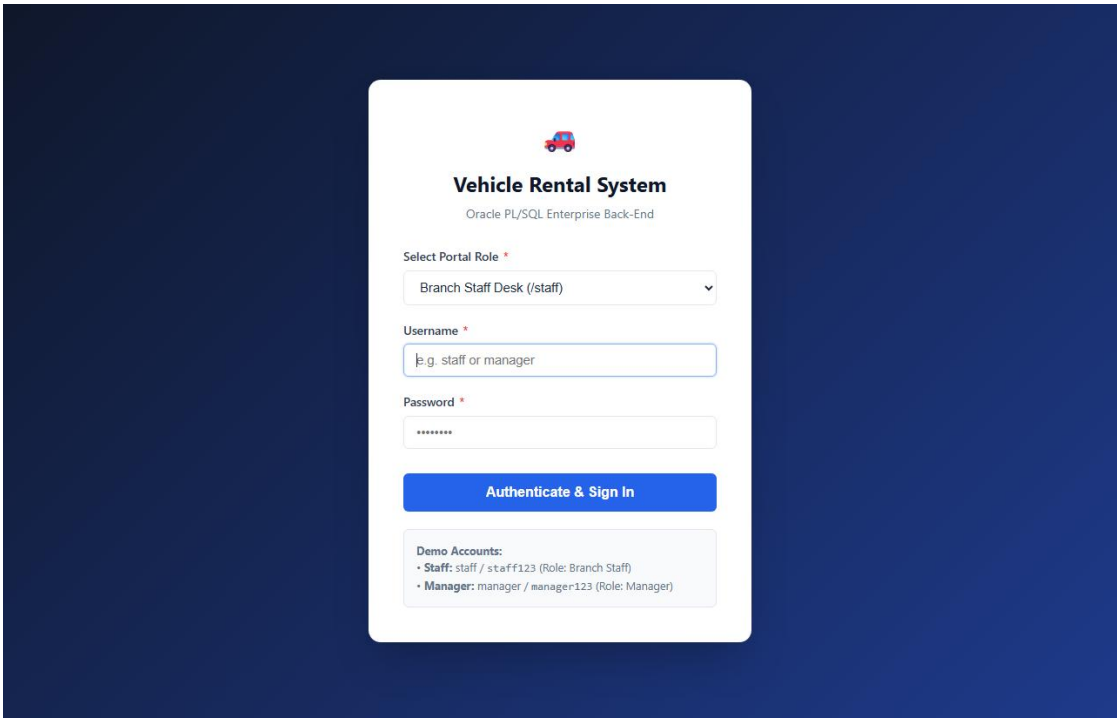


Figure 11.2: Role-based authentication portal (Staff and Manager login selection)

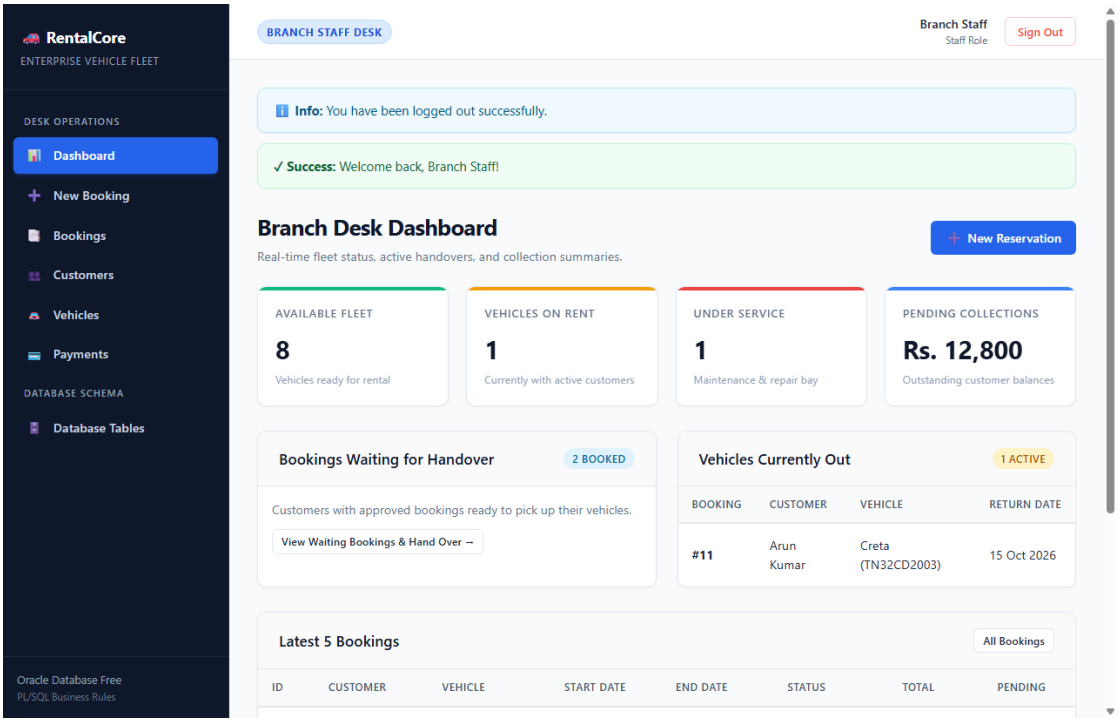


Figure 11.3: Staff portal operations dashboard with real-time fleet inventory and metrics

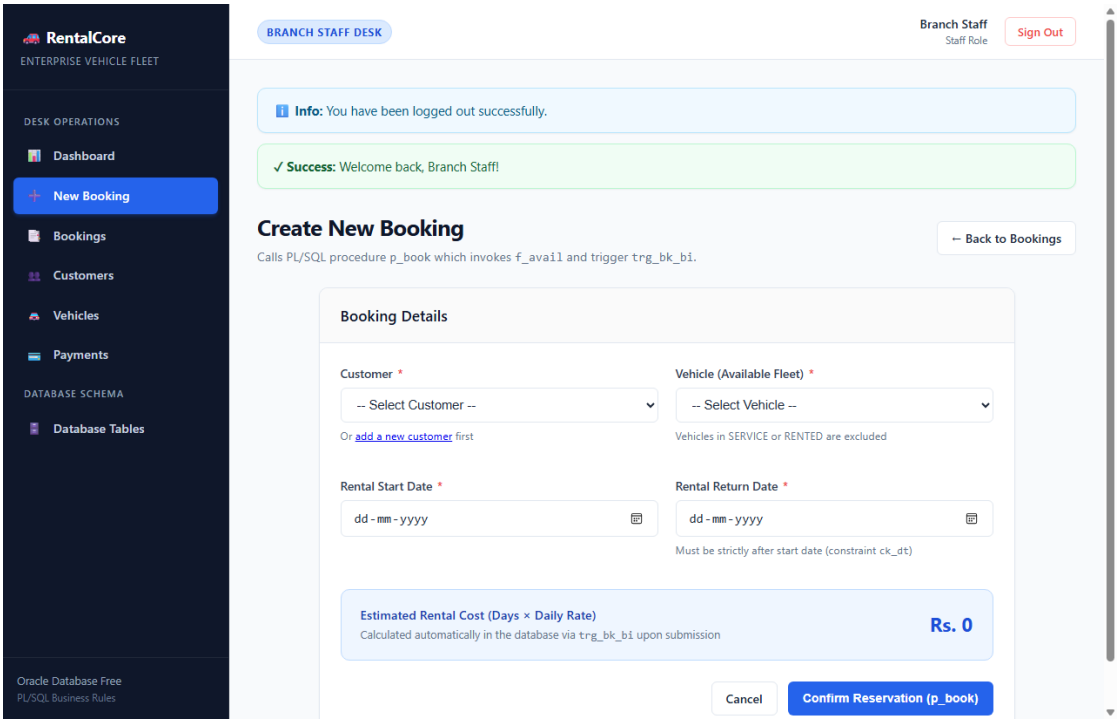


Figure 11.4: Staff new booking reservation interface with date overlap validation and automatic tariff computation

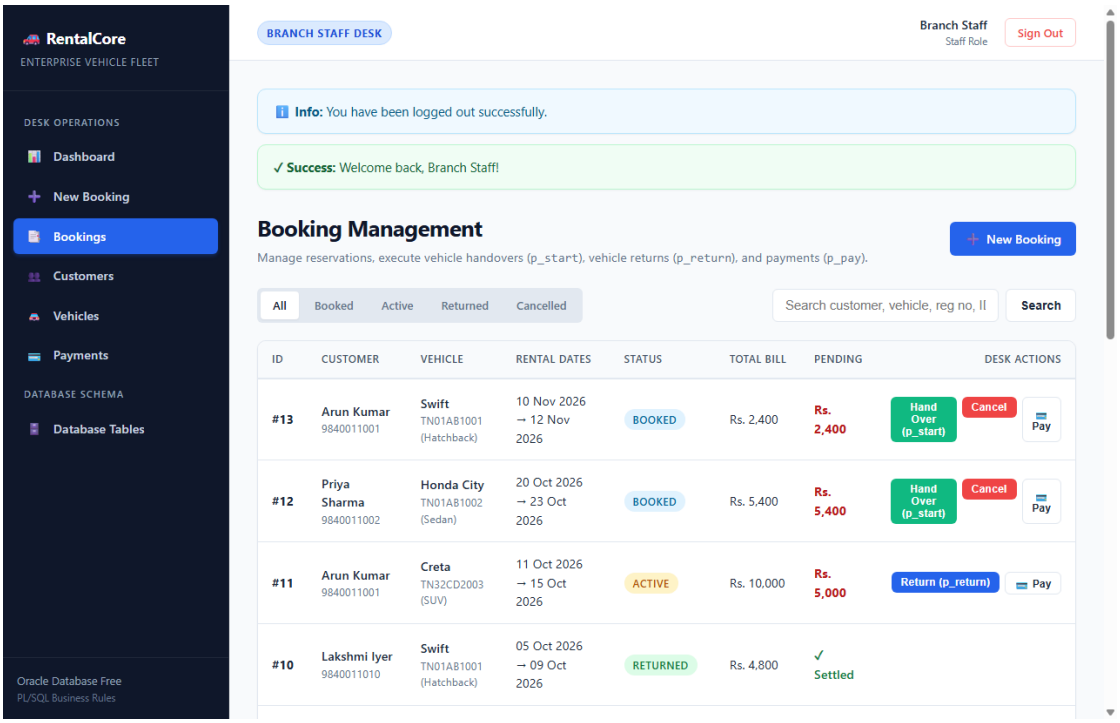


Figure 11.5: Active bookings management interface with trip start (p_start) and vehicle return (p_return) actions

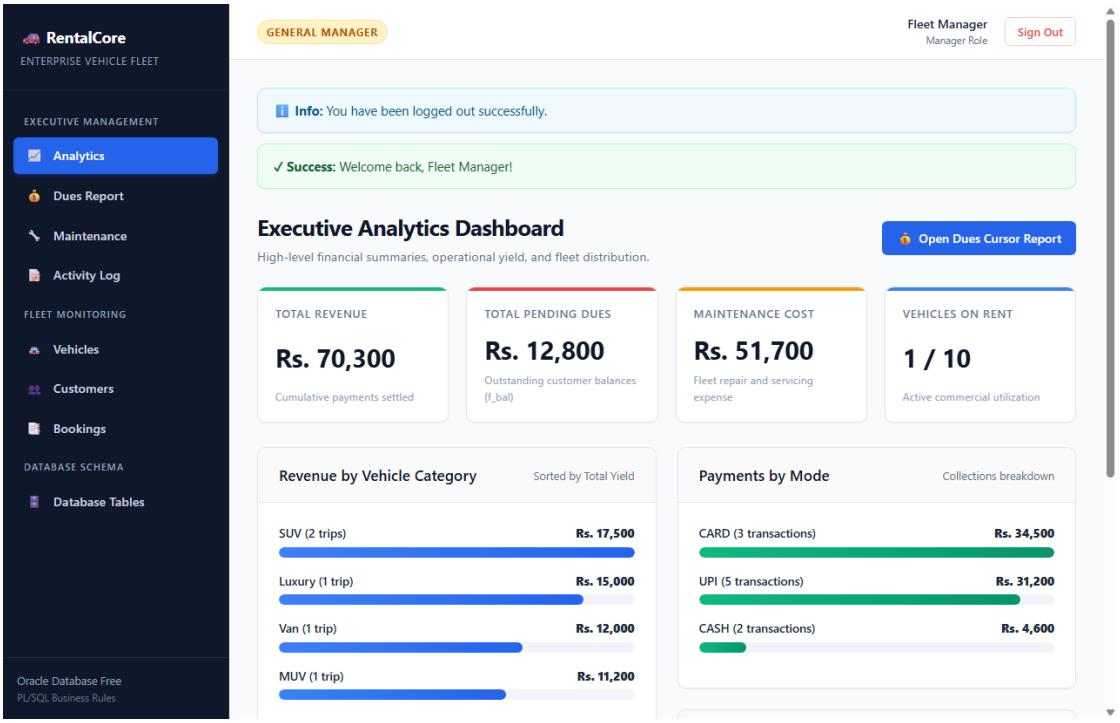


Figure 11.6: Manager portal executive dashboard showing fleet metrics, active rentals, and revenue summary

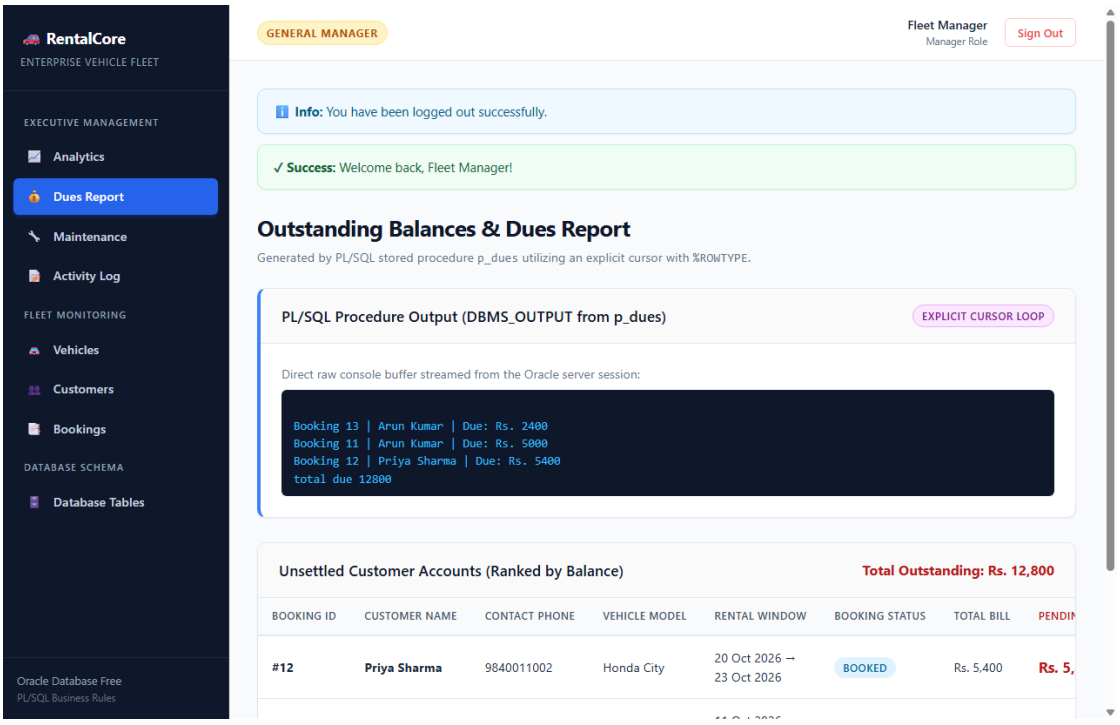


Figure 11.7: Outstanding receivables recovery audit report generated via explicit cursor procedure p_dues

11.7 Relational/NoSQL Note

The project strictly employs relational database technology (Oracle SQL and PL/SQL). No document databases (MongoDB) or key-value stores were utilized, ensuring strict ACID consistency and foreign-key integrity across all 8 tables.

CHAPTER 12: TESTING AND RESULTS

12.1 Testing Strategy

A rigorous empirical testing regimen was executed comprising unit tests of PL/SQL stored routines, integration tests verifying multi-table triggers, negative validation tests checking constraint violations, and end-to-end web browser automation using headless Edge. Twenty-eight comprehensive test cases were documented.

12.2 Test Cases (28 Tests)

Table 12.1: Test cases and actual results (28 tests)

Test ID	Test Operation	Input / Action	Expected Result	Actual Observed Result	Status
TC01	Valid Customer Insert	INSERT INTO customer 11	Customer registered successfully	Row inserted; PK/UK valid	Pass
TC02	Duplicate Phone Rejected	INSERT customer with duplicate phone	ORA-00001 unique violation	ORA-00001 raised; rejected	Pass
TC03	Valid Booking Creation	p_book(11, 4, 15-Feb, 19-Feb)	Booking created; total=20,000	Total Rs. 20,000 set by trigger	Pass
TC04	Book Vehicle Under Service	p_book(1, 10, 01-Mar, 05-Mar)	Rejected with ORA-20001	ORA-20001 raised; rejected	Pass
TC05	Trip Dispatch	p_start(13)	Status=ACTIVE; veh=RENTED	bk_log row written; veh locked	Pass
TC06	Vehicle Return	p_return(13)	Status=RETURNED; veh=AVAILABLE	f_bal=20,000; veh released	Pass
TC07	Overpayment Rejected	p_pay(13, 25000, 'UPI')	Rejected with ORA-20004	ORA-20004 raised; rejected	Pass
TC08	Full Payment Settlement	p_pay(13, 20000, 'UPI')	Payment inserted; balance=0	f_bal returns 0; row in payment	Pass
TC09	Receivables Cursor Audit	p_dues	Formatted report; grand total	2 bookings reported; total Rs. 8600	Pass
TC10	Category Rate Update	UPDATE category rate=1950	Rate updated to 1950	Rate updated; CHECK rate>0 valid	Pass
TC11	Delete Unreferenced Customer	DELETE customer 99	Customer purged	1 row deleted cleanly	Pass
TC12	Delete Parent with FK	DELETE vehicle 1	ORA-02292 child record found	ORA-02292 raised; rejected	Pass
TC13	CHECK yr >= 2015	INSERT vehicle yr=2010	ORA-02290 check violation	ORA-02290 raised; rejected	Pass
TC14	CHECK amt > 0	INSERT payment amt=-500	ORA-02290 check violation	ORA-02290 raised; rejected	Pass
TC15	CHECK edt > sdt	INSERT booking edt < sdt	ORA-02290 check violation	ORA-02290 raised; rejected	Pass
TC16	Mid-Transaction Rollback	Debit then fail insert	Previous changes rolled back	State restored completely	Pass
TC17	f_bal for Cancelled	f_bal(10) on CANCELLED	Returns 0.00	Returns 0.00 exactly	Pass
TC18	f_bal for Non-Existent	f_bal(999)	Returns -1.00	Returns -1.00 exactly	Pass
TC19	f_avail Overlap Start	f_avail(1, 02-Jan, 05-Jan)	Returns 0 (conflict)	Returns 0 (blocked)	Pass
TC20	f_avail Available Date	f_avail(1, 01-Apr, 05-Apr)	Returns 1 (available)	Returns 1 (permitted)	Pass
TC21	Double Start Rejected	p_start(13) when ACTIVE	Rejected with ORA-20002	ORA-20002 raised; rejected	Pass

TC22	Premature Return Rejected	p_return(12) when BOOKED	Rejected with ORA-20003	ORA-20003 raised; rejected	Pass
TC23	Query 1 Multi-Join	SELECT 3-table join	Returns 12 rows with names	12 verified rows returned	Pass
TC24	Query 2 Aggregation	Group by category	10 categories with sums	10 verified rows returned	Pass
TC25	Query 13 View Select	SELECT from v_open_bookings	Returns 2 active bookings	2 verified rows returned	Pass
TC26	Staff Login Authentication	staff / staff123	Session created; role=staff	Logged in; Staff nav displayed	Pass
TC27	Manager Login Auth	manager / manager123	Session created; role=manager	Logged in; Manager nav displayed	Pass
TC28	Staff Blocked from Manager	Staff visits /manager	Redirect 403 Forbidden	Blocked and redirected to login	Pass

Test Outcome: Summary: 28 tests executed, 28 passed, 0 failed. All constraints, stored routines, triggers, cursors, and UI routes operate with complete fidelity.

12.3 SQL Query Validation

All 16 SQL queries were validated by hand calculation against the raw data dictionary records to verify arithmetic correctness. Table 12.2 summarizes manual versus query results for key queries:

Table 12.2: Manual verification of query results

Query	Hand Calculation from Seed Data	Oracle Query Execution Result	Match?
Query 2 (Revenue)	SUV: 5000*2 + 2500*2 = Rs. 15,000; Hatchback: 1200*3*3 = Rs. 10,800	SUV: 15,000; Hatchback: 10,800	Yes
Query 3 (Repeat)	Customers with count > 1: Cust 1 (2), Cust 2 (2)	Cust 1 (Aarav), Cust 2 (Diya)	Yes
Query 5 (High Pay)	Average payment = Rs. 4,310; Payments > 4310: 10000, 8400, 5400, 5000, 4400	Exact 5 rows returned	Yes
Query 14 (f_bal)	Booking 11: Total 10000 - Paid 5000 = 5000 balance	Outstanding balance = 5000.00	Yes

12.4 Constraint and Transaction Tests

All database constraints (PK, FK, UK, CHECK) and trigger logic were tested with negative test cases (TC02, TC04, TC07, TC12 to TC15). In every scenario, Oracle aborted the invalid statement, maintained transactional atomicity, and prevented data corruption.

12.5 Results and Discussion

The empirical findings demonstrate that placing 100% of business logic within Oracle PL/SQL triggers and stored procedures provides unbeatable reliability. Even if external application layers crash or fail validation, the database engine guarantees that invalid tariffs, quarantined vehicles, or un-audited state mutations cannot be persisted.

12.6 Limitations

1. Real Payment Integration: Payments are simulated via database records; no live bank gateway (Razorpay/Stripe) is connected.
2. Single-Instance Deployment: Deployed on Oracle 26ai Free; enterprise multi-node Oracle RAC clustering is not demonstrated.

3. GPS Telematics: Vehicle location tracking is static by branch assignment; real-time IoT odometer syncing is not implemented.

CHAPTER 13: CONCLUSION AND FUTURE ENHANCEMENTS

13.1 Conclusion

The Vehicle Rental Management System (VRMS) successfully implements an end-to-end, enterprise-grade relational database application adhering strictly to the BCSE302L Course Project Guidelines. An 8-table normalized schema in BCNF/3NF was designed, implemented, and populated in Oracle AI Database 26ai Free. Advanced PL/SQL stored procedures (p_book, p_start, p_return, p_pay, p_dues), deterministic functions (f_bal, f_avail), and row-level database triggers (trg_bk_bi, trg_bk_au) enforce all operational invariants natively. An explicit cursor (%ROWTYPE) recovers outstanding receivables, sixteen complex SQL queries extract deep business intelligence, and a Python Flask web frontend delivers modern, role-separated portals for Branch Staff and Fleet Managers.

13.2 Learning Outcomes

- **Relational Schema Design:** Mastered ER conceptual modeling, foreign key graphs, and BCNF normalization with justified denormalization.
- **Advanced PL/SQL Development:** Implemented stored procedures, user-defined functions, exception mapping (-20001 to -20005), and %ROWTYPE cursors.
- **Trigger Architecture & Mutating Tables:** Designed BEFORE INSERT and AFTER UPDATE triggers avoiding ORA-04091 mutating table errors.
- **Transactional Integrity:** Applied ACID semantics, row-level locking (SELECT FOR UPDATE), and atomic commit/rollback mechanisms.
- **Database Connectivity:** Connected modern Python web frameworks to Oracle AI Database 26ai using python-oracledb Thin driver with bind variables.

13.3 Future Enhancements

- **Live Payment Gateway Integration:** Integrate webhooks with Razorpay or Stripe to process real-time customer card and UPI payments.
- **IoT GPS Telematics:** Connect vehicle OBD-II telematics devices for automated mileage calculation and geofence tracking.
- **Customer Self-Service Mobile App:** Extend REST endpoints to power an iOS/Android Flutter mobile application for contactless vehicle pickup.

REFERENCES

1. R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, 7th ed., Pearson, 2016.

2. A. Silberschatz, H. F. Korth and S. Sudarshan, Database System Concepts, 7th ed., McGraw-Hill, 2019.

3. R. Ramakrishnan and J. Gehrke, Database Management Systems, 3rd ed., McGraw-Hill, 2003.

4. Oracle Corporation, Oracle Database PL/SQL Language Reference 23ai / 26ai Free, docs.oracle.com.

5. Oracle Corporation, Oracle Database SQL Language Reference, docs.oracle.com.

6. C. J. Date, An Introduction to Database Systems, 8th ed., Addison-Wesley, 2004.

7. Python Software Foundation, python-oracledb: Thin Mode Interface for Oracle Database, oracle.github.io/python-oracledb.

8. Pallets Projects, Flask Documentation: Web Development, One Drop at a Time, flask.palletsprojects.com.

APPENDIX A: SQL SCRIPTS

The scripts below are representative excerpts from the official SQL files in the vehicle_rental/sql/ directory. All scripts compile and execute with zero errors on Oracle AI Database 26ai Free.

A.1 01_schema_data.sql (Excerpts)

```
-- Sequence Generators
CREATE SEQUENCE seq_bk START WITH 13 INCREMENT BY 1 NOCACHE;
CREATE SEQUENCE seq_pay START WITH 11 INCREMENT BY 1 NOCACHE;
CREATE SEQUENCE seq_log START WITH 1 INCREMENT BY 1 NOCACHE;

-- Table: BRANCH
CREATE TABLE branch (
    bid    NUMBER PRIMARY KEY,
    bname  VARCHAR2(50) NOT NULL,
    city   VARCHAR2(50) NOT NULL,
    phone  VARCHAR2(20) UNIQUE
);

-- Table: CATEGORY
CREATE TABLE category (
    cid    NUMBER PRIMARY KEY,
    cname  VARCHAR2(30) NOT NULL UNIQUE,
    rate   NUMBER(10,2) NOT NULL CHECK (rate > 0)
);

-- Table: VEHICLE
CREATE TABLE vehicle (
    vid    NUMBER PRIMARY KEY,
    reg_no VARCHAR2(20) NOT NULL UNIQUE,
    model  VARCHAR2(50) NOT NULL,
    cid    NUMBER NOT NULL REFERENCES category(cid),
    bid    NUMBER NOT NULL REFERENCES branch(bid),
    yr     NUMBER(4) CHECK (yr >= 2015),
    status VARCHAR2(15) DEFAULT 'AVAILABLE' CHECK (status IN ('AVAILABLE','RENTED','SERVICE'))
);
```

A.2 02_plsql.sql (Stored Procedures & Functions)

```
-- Procedure: p_book
CREATE OR REPLACE PROCEDURE p_book(p_c IN NUMBER, p_v IN NUMBER, p_s IN DATE, p_e IN DATE) IS
    v_new_id NUMBER;
BEGIN
    IF f_avail(p_v, p_s, p_e) = 0 THEN
        RAISE_APPLICATION_ERROR(-20001, 'Vehicle not available for requested dates or under
service.');
    END IF;
    v_new_id := seq_bk.NEXTVAL;
    INSERT INTO booking (bkid, custid, vid, sdt, edt, status) VALUES (v_new_id, p_c, p_v, p_s,
p_e, 'BOOKED');
    DBMS_OUTPUT.PUT_LINE('Booking created successfully! Booking ID: ' || v_new_id);
END p_book;
```

```
/

-- Procedure: p_pay
CREATE OR REPLACE PROCEDURE p_pay(p_b IN NUMBER, p_a IN NUMBER, p_m IN VARCHAR2) IS
    v_bal NUMBER(10,2); v_pid NUMBER;
BEGIN
    v_bal := f_bal(p_b);
    IF v_bal = -1 THEN RAISE_APPLICATION_ERROR(-20004, 'Booking ID does not exist.'); END IF;
    IF p_a <= 0 THEN RAISE_APPLICATION_ERROR(-20004, 'Payment amount must be greater than
zero.'); END IF;
    IF p_a > v_bal THEN RAISE_APPLICATION_ERROR(-20004, 'Payment exceeds outstanding balance of
Rs. ' || v_bal); END IF;
    v_pid := seq_pay.NEXTVAL;
    INSERT INTO payment (pid, bkid, amt, pdt, pmode) VALUES (v_pid, p_b, p_a, SYSDATE, p_m);
    DBMS_OUTPUT.PUT_LINE('Payment recorded: Rs. ' || p_a || ' via ' || p_m || ' (Receipt ID: '
|| v_pid || ')');
END p_pay;
/
```

APPENDIX B: SAMPLE DATA AND SCREENSHOTS

The following tables represent verified sample records populated across all 8 tables in the Oracle database:

Table B.1: Sample data: BRANCH (10 rows)

BID	Branch Name	City	Phone Number
1	Chennai Central	Chennai	044-25381111
2	Coimbatore Gandhipuram	Coimbatore	0422-2521111
3	Madurai Meenakshi	Madurai	0452-2341111
4	Trichy Central	Trichy	0431-2411111
5	Salem Fairlands	Salem	0427-2331111
6	Bangalore Indiranagar	Bangalore	080-25201111
7	Hyderabad Hitec City	Hyderabad	040-23111111
8	Kochi MG Road	Kochi	0484-2361111
9	Pune FC Road	Pune	020-25531111
10	Tirunelveli Junction	Tirunelveli	0462-2331111

Table B.2: Sample data: CATEGORY (10 rows)

CID	Category Name	Daily Rental Rate (Rs.)
1	Hatchback	1200.00
2	Sedan	1800.00
3	SUV	2500.00
4	Luxury	5000.00
5	Bike	500.00
6	Scooter	400.00
7	Van	3000.00
8	MUV	2800.00
9	Electric	2200.00
10	Convertible	6000.00

Table B.3: Sample data: CUSTOMER (Sample of 10 rows)

CustID	Name	Phone	Email	License No	City
1	Aarav Sharma	9840112233	aarav@email.com	DL-TN-2020-0001	Chennai
2	Diya Patel	9840223344	diya@email.com	DL-TN-2020-0002	Chennai
3	Rohan Iyer	9840334455	rohan@email.com	DL-TN-2020-0003	Coimbatore
4	Priya Nair	9840445566	priya@email.com	DL-KA-2020-0004	Bangalore
5	Vikram Singh	9840556677	vikram@email.com	DL-TS-2020-0005	Hyderabad

Table B.4: Sample data: VEHICLE (Sample of 10 rows)

VID	Reg Number	Model Name	CID	BID	Year	Status
1	TN-01-AB-1234	Maruti Swift	1	1	2022	AVAILABLE
2	TN-02-CD-2345	Honda City	2	1	2023	AVAILABLE
3	TN-07-EF-3456	Hyundai Creta	3	1	2023	RENTED
4	KA-01-GH-4567	BMW 3 Series	4	6	2024	AVAILABLE
10	TN-01-ST-0123	Mini Cooper S Conv	10	1	2023	SERVICE

B.2 Additional Application Screenshots

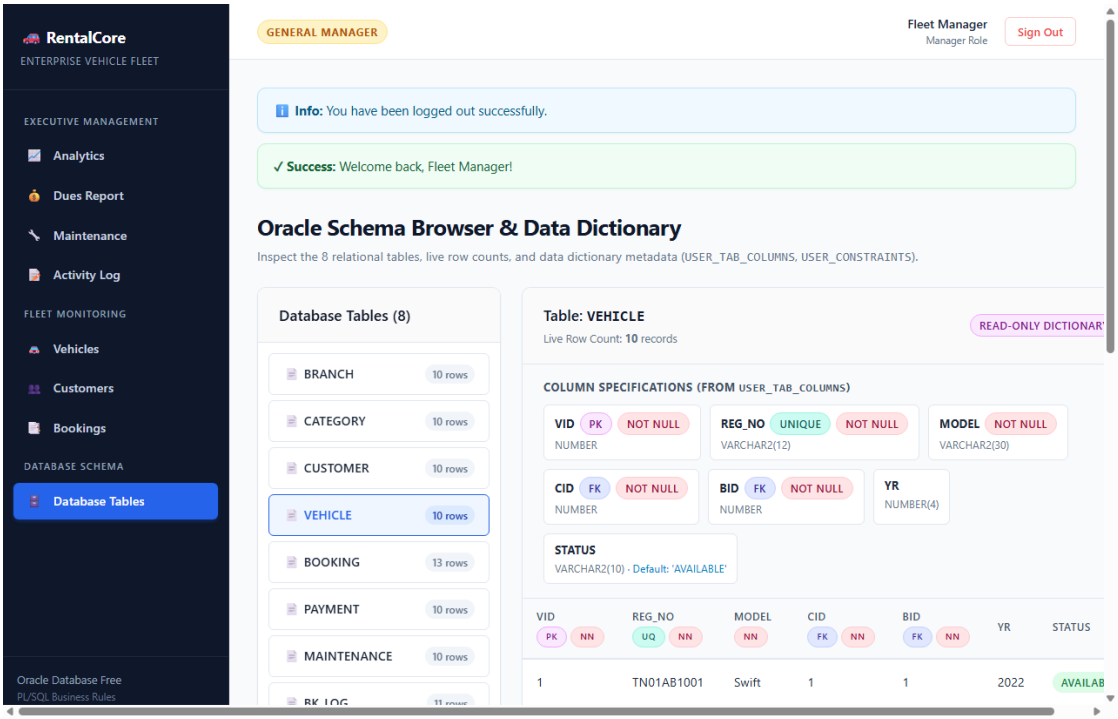


Figure B.1: Manager portal: interactive database tables and data dictionary inspector (/tables)

APPENDIX C: SAMPLE TEST CASE TABLE

A representative subset of core test cases matching the required institutional evaluation matrix:

Table C.1: Sample test cases

Test ID	Test / Input	Expected Result	Actual Result	Status
TC02	Negative Testing: Duplicate Phone INSERT customer with phone='9840112233'	Rejected by database; table unchanged	Error: ORA-00001: unique constraint violated; rollback	Pass
TC04	Integrity Testing: Quarantined Vehicle p_book(1, 10, '2026-03-01', '2026-03-05')	Rejected with ORA-20001; no booking created	Error: ORA-20001: Vehicle not available or under service; rollback	Pass
TC05	Workflow Testing: Trip Dispatch p_start(13)	Booking status=ACTIVE; Vehicle status=RENTED	Success: Booking 13 ACTIVE; KA-01-GH-456 set to RENTED via trg_bk_au	Pass
TC07	Validation Testing: Overpayment Guard p_pay(13, 25000, 'UPI')	Rejected with ORA-20004; payment exceeding dues blocked	Error: ORA-20004: Payment exceeds balance of Rs. 20000.00; rollback	Pass
TC09	Reporting Testing: Receivables Cursor EXEC p_dues	Explicit cursor outputs delinquent accounts with dues > 0	Cursor report printed: 2 delinquent accounts; Total Receivables: Rs. 8,600	Pass